

Memory Profiling for Greener Software in Dynamically-Typed Languages

Sebastian JORDAN MONTAÑO

PhD Defense

Supervisors

Guillermo POLITO

Stéphane DUCASSE

Jury

Gaël THOMAS

Tobias WRINGSTAD

Gordana RAKIC



Inria



 **Université
de Lille**

 **centralelille**
ÉCOLE CENTRALE DE LILLE

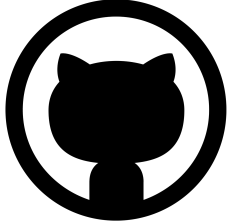
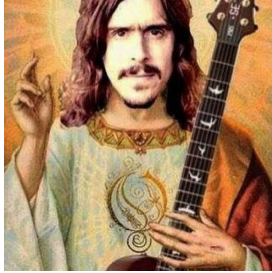
 **CRISTAL**
Centre de Recherche en Informatique,
Signal et Automatique de Lille


**Région
Hauts-de-France**



Sep 2026, Lille, France

Hello, I am Sebastian

- I am from Cochabamba, Bolivia
- I am Software Engineer
- I am an open-source contributor since 2020
- Check me out on  at `jordanmontt`
- I like Progressive Death Metal 🤘🎸 
- King Crimson = best progressive rock group 🎸🎵

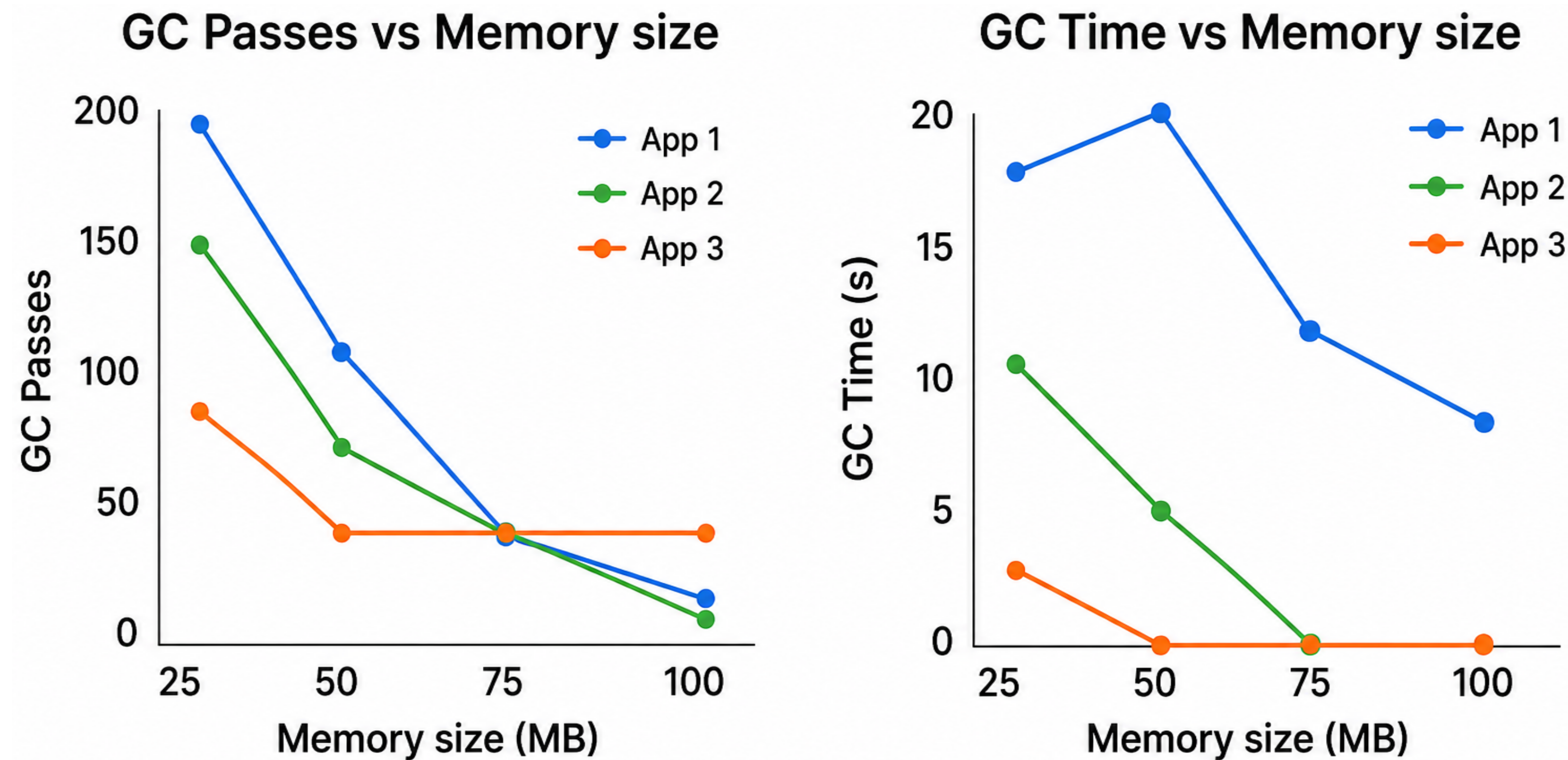


Pathological Memory Behavior

Application	GC overhead
App 1	15%
App 2	24%
App 3	89% 🤯



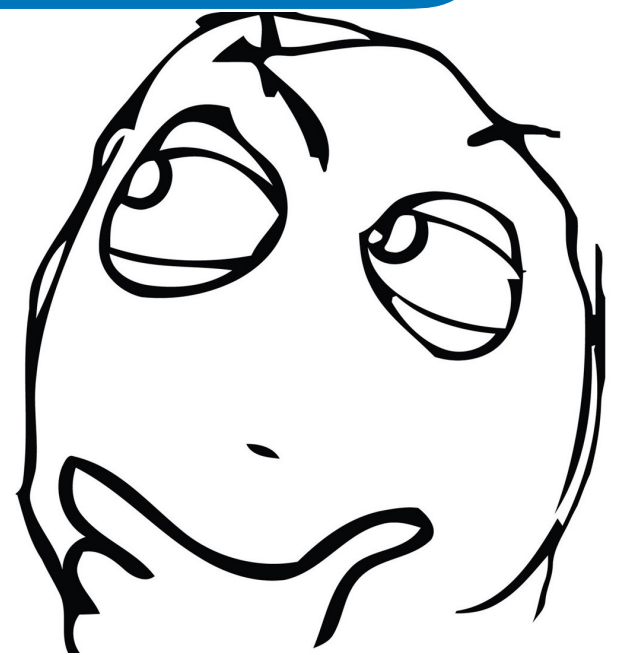
Different GC Behavior, Same Parameters



Behavior is application-dependant

Thesis Goal

How can we make memory behavior visible to apply memory optimizations?



Outline

1. Introduction
- 2. State of the Art**
3. Finalization-Based Lifetime Profiling
4. Textual Allocation Site Pretenuring
5. Path-Sensitive Pretenuring
6. Conclusion

Context: Pharo

- Pure object-oriented language
- Allocations are done through normal method invocation
- Mature VM, stable in production +10 years
- Generation Garbage Collector (GC)



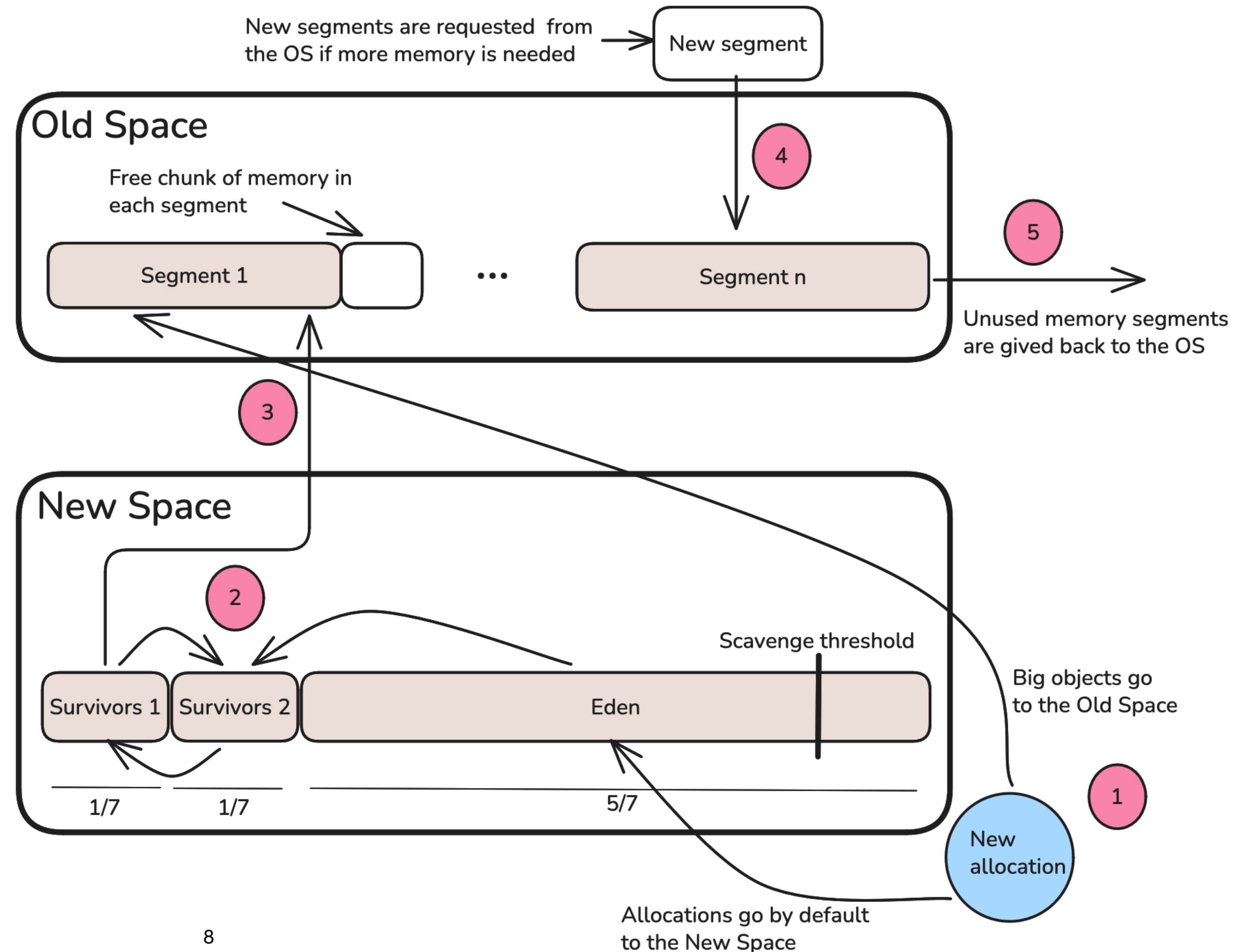
Generational GC in Pharo

Heap structure:

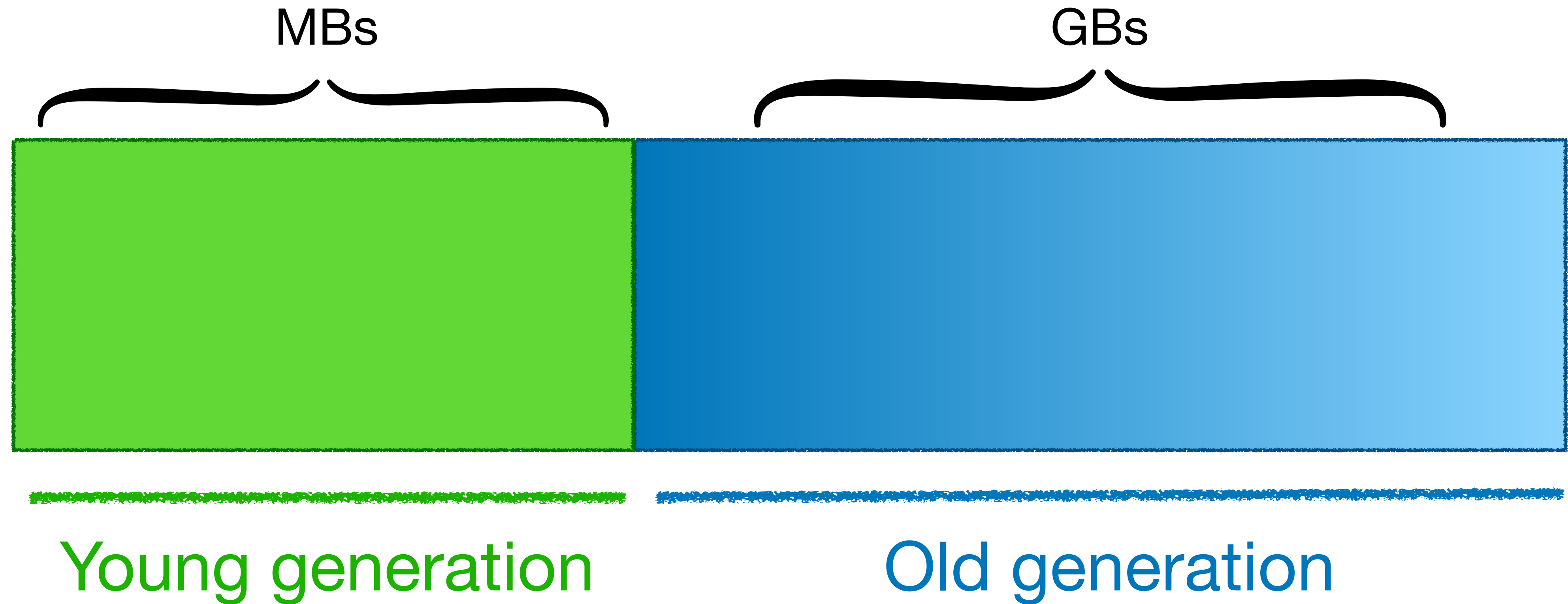
- **New Space:** Eden + 2 semi-spaces (survivors)
- **Old Space:** larger, collected occasionally

Collection algorithms:

- Scavenger (copy-collector) for New Space
- Mark-compact for Old Space



Generational Garbage Collectors



Weak generational hypothesis

Most objects die young 🦴🐣🔫

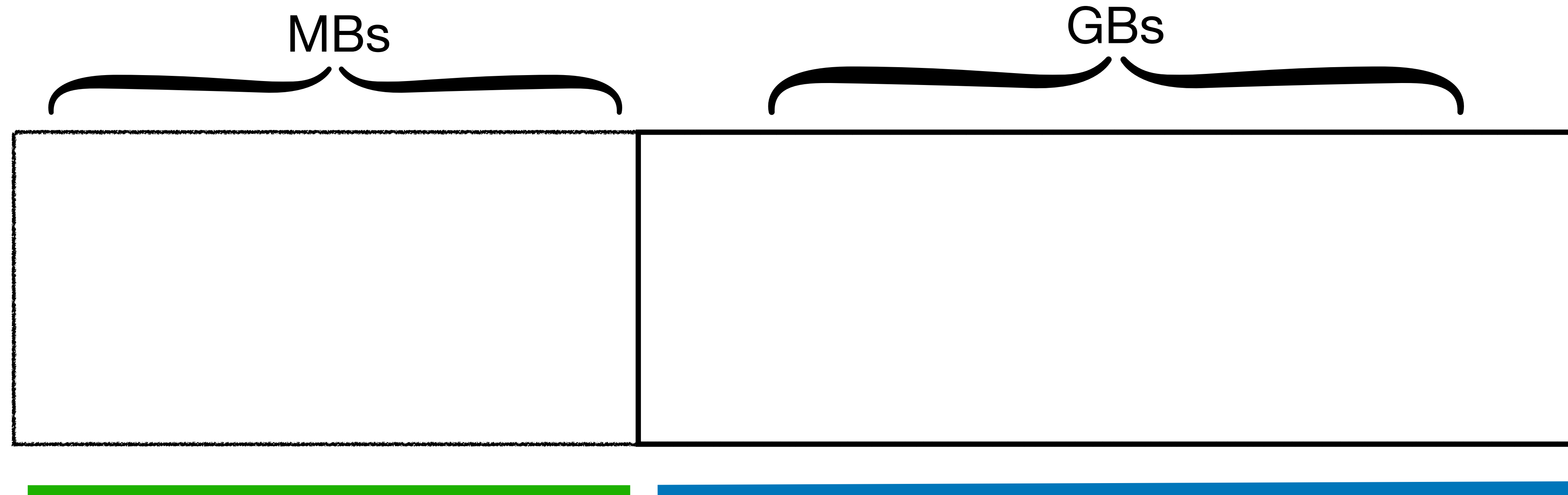
- Majority of objects die immediately after being allocated
- Objects are organized by lifetime
- Memory is organized in at least two generations (young and olds)
- Most mainstream languages implement this collector

**Generation Scavenging: A Non-disruptive High Performance
Storage Reclamation Algorithm**

David Ungar

Computer Science Division
Department of Electrical Engineering and Computer Sciences
University of California
Berkeley, California 94720

Generational Garbage Collectors



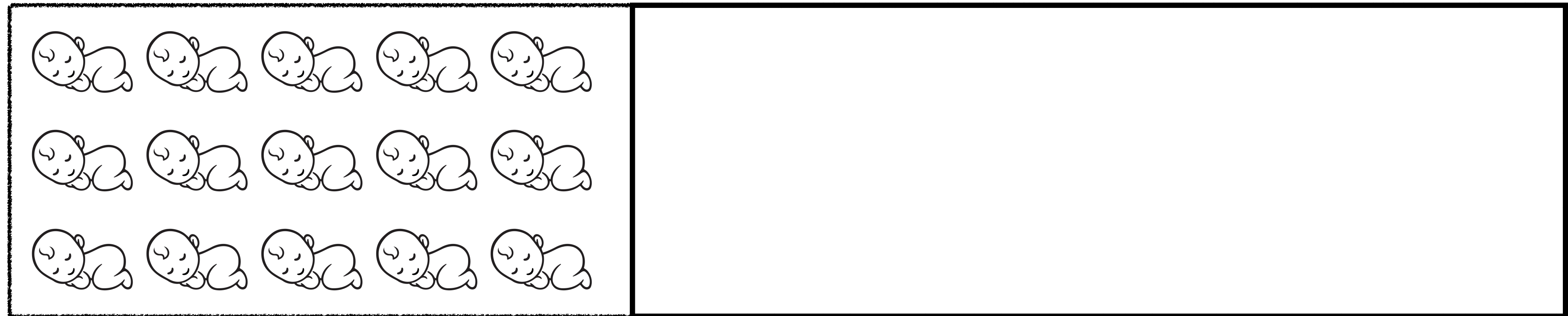
Young generation

- Smaller space
- Frequent and faster collections

Old generation

- Bigger space
- Rarer and slower collections

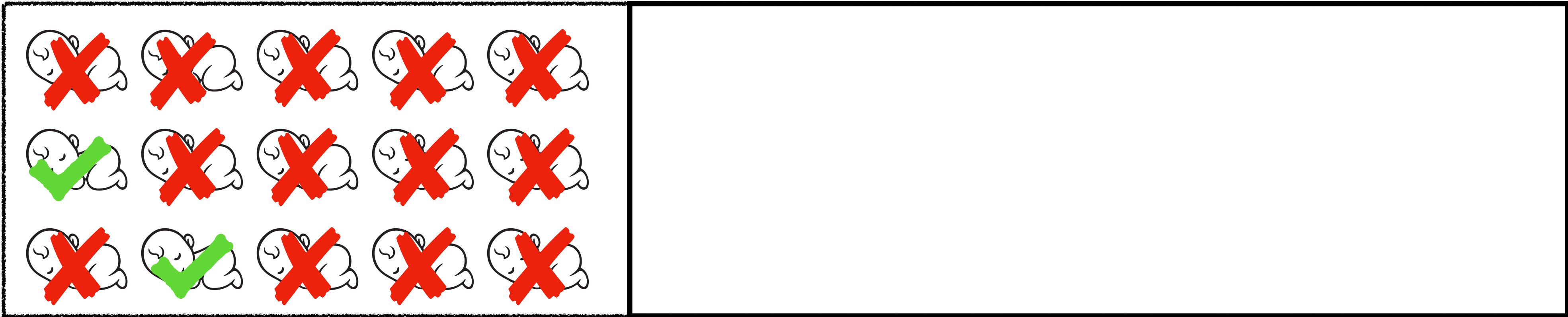
When One Allocates Objects



Young generation

Old generation

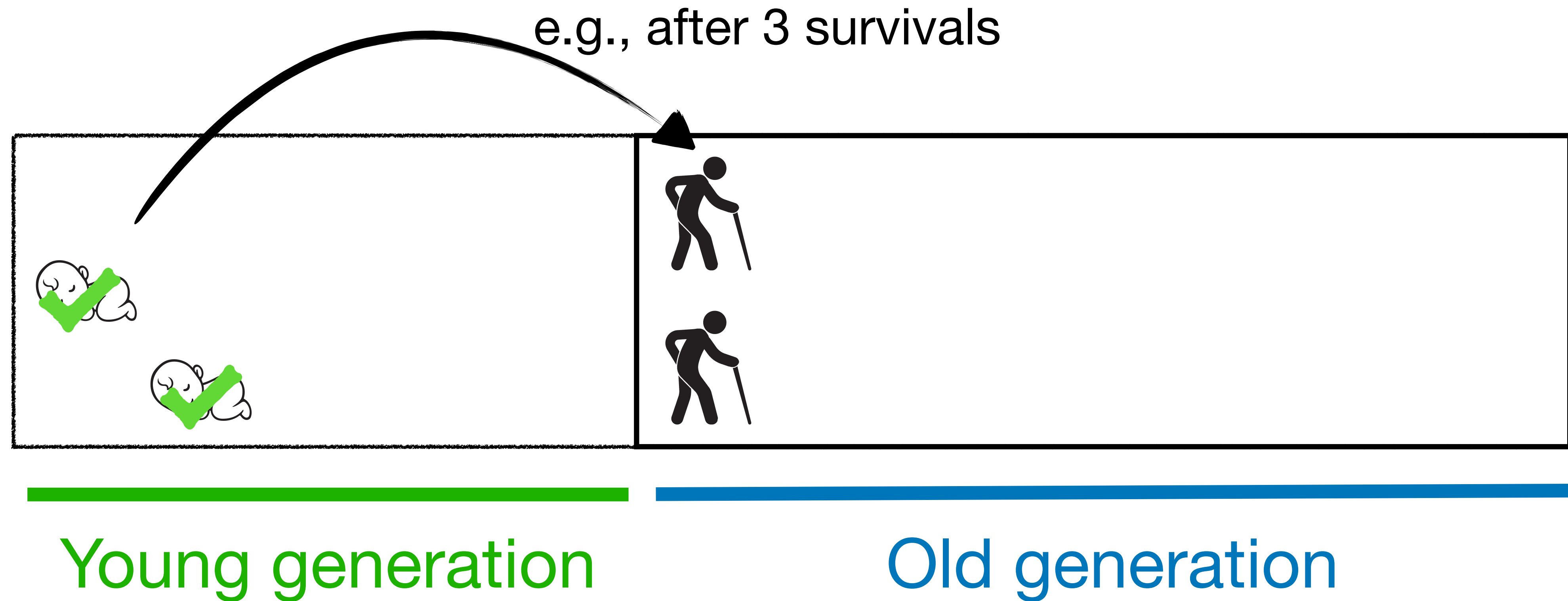
Few Objects Survive



Young generation

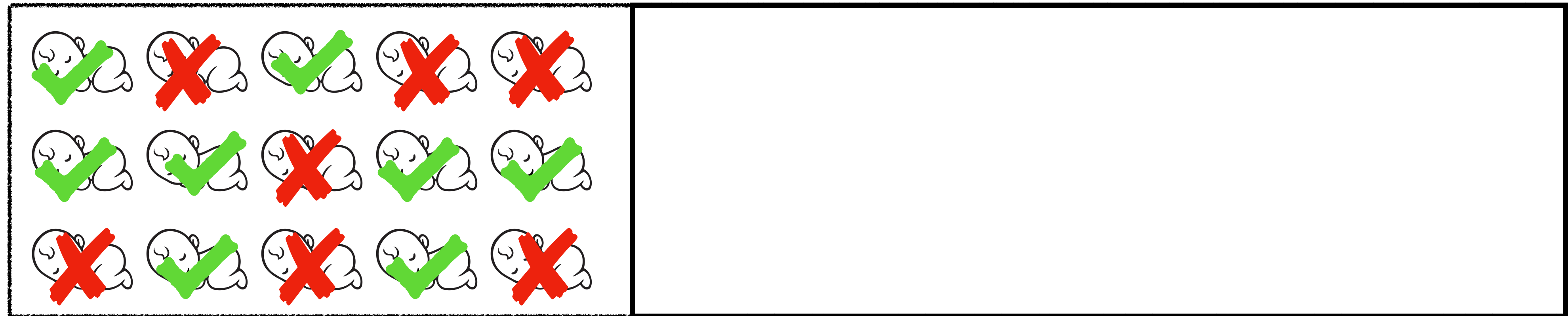
Old generation

Tenuring After Several Generations



* Tenured = promoted to the old generation

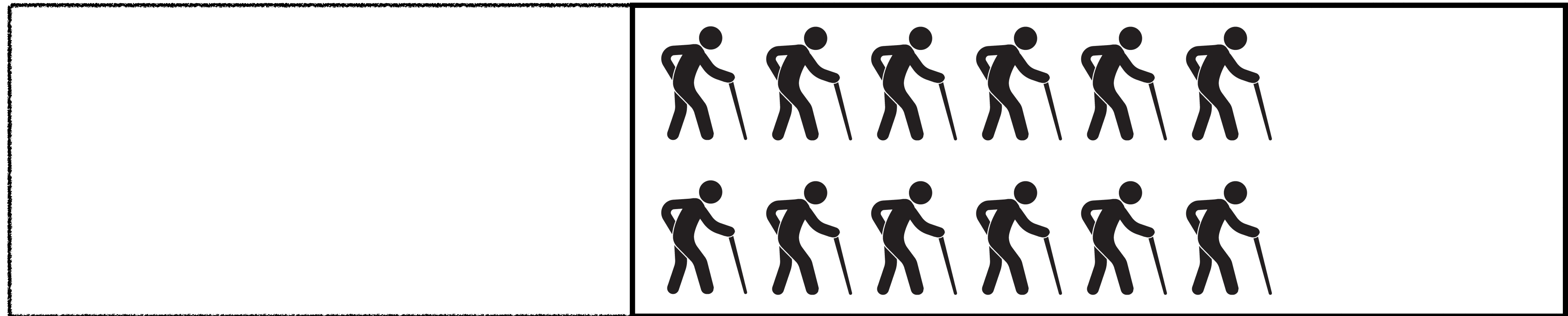
Some Applications Behave Differently



Young generation

Old generation

The GC Is Under Heavy Pressure



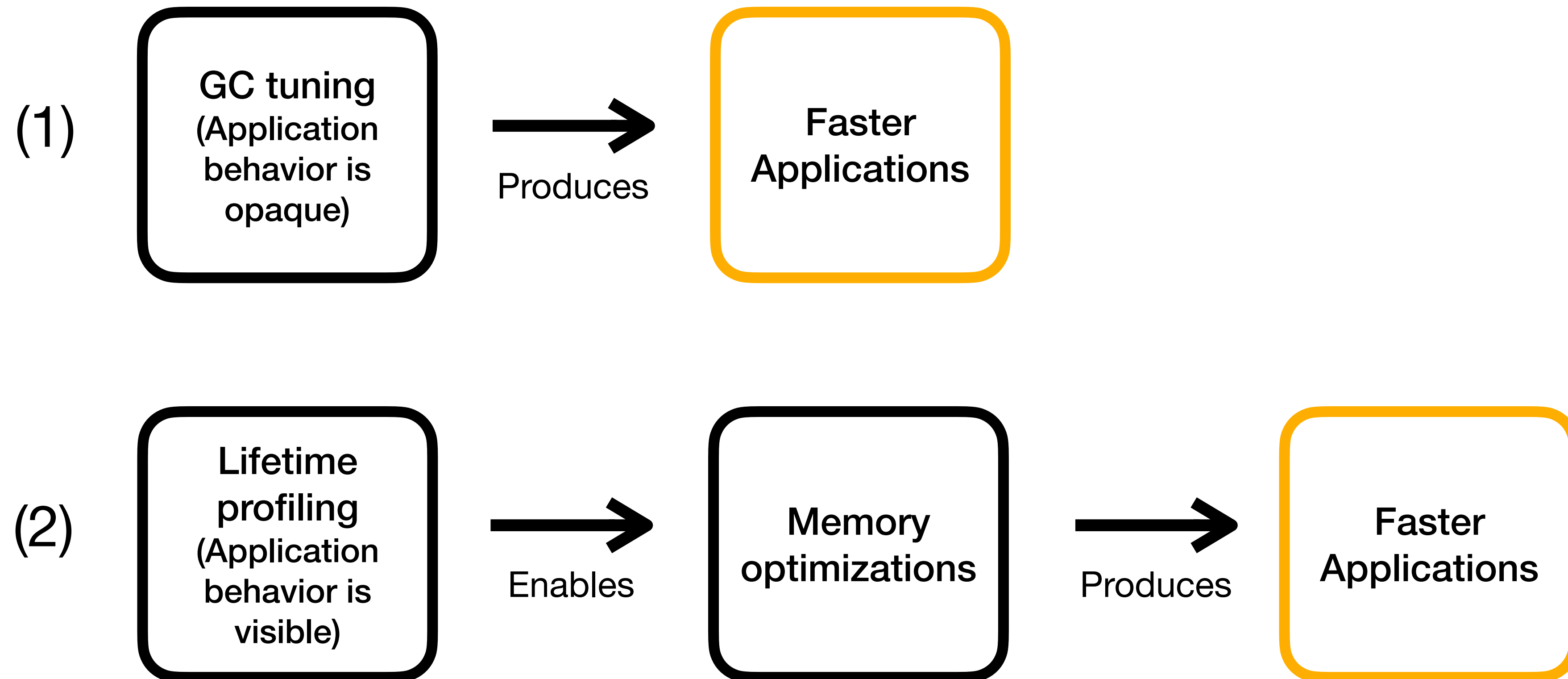
Young generation

Old generation

More Time is Spent Managing Memory than Executing Code



Two Possible Solutions

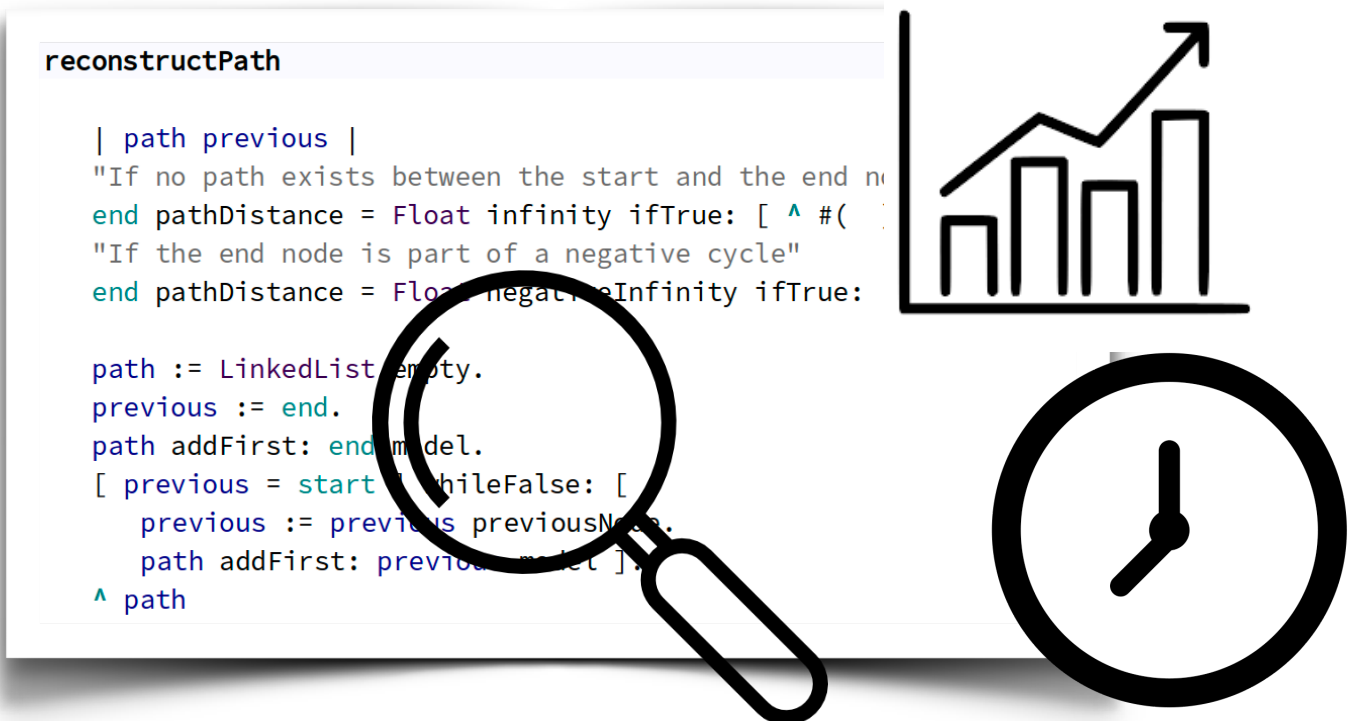


Black Box Tuning

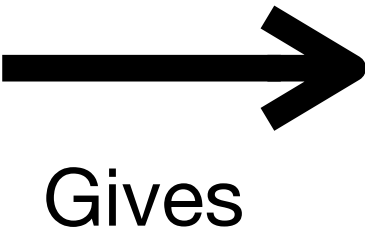
- Parameters control the memory sizes and the criteria to trigger collections.
- GCs expose many parameters (JDK ~280). Defaults are rarely optimal ★
- Optimizations algorithms (genetic, hill climbing) ★

Example: DataFrame application		
Configuration	GC Overhead	Speedup
Default parameters	89%	—
Tuned parameters	18.4%	7.33×

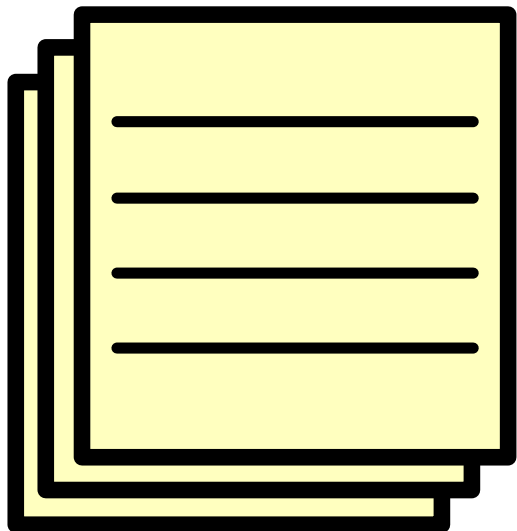
Profile-Guided Pretenuring



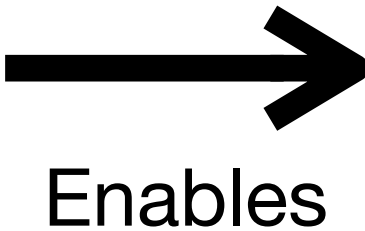
Profiler



Gives



Object lifetime
information

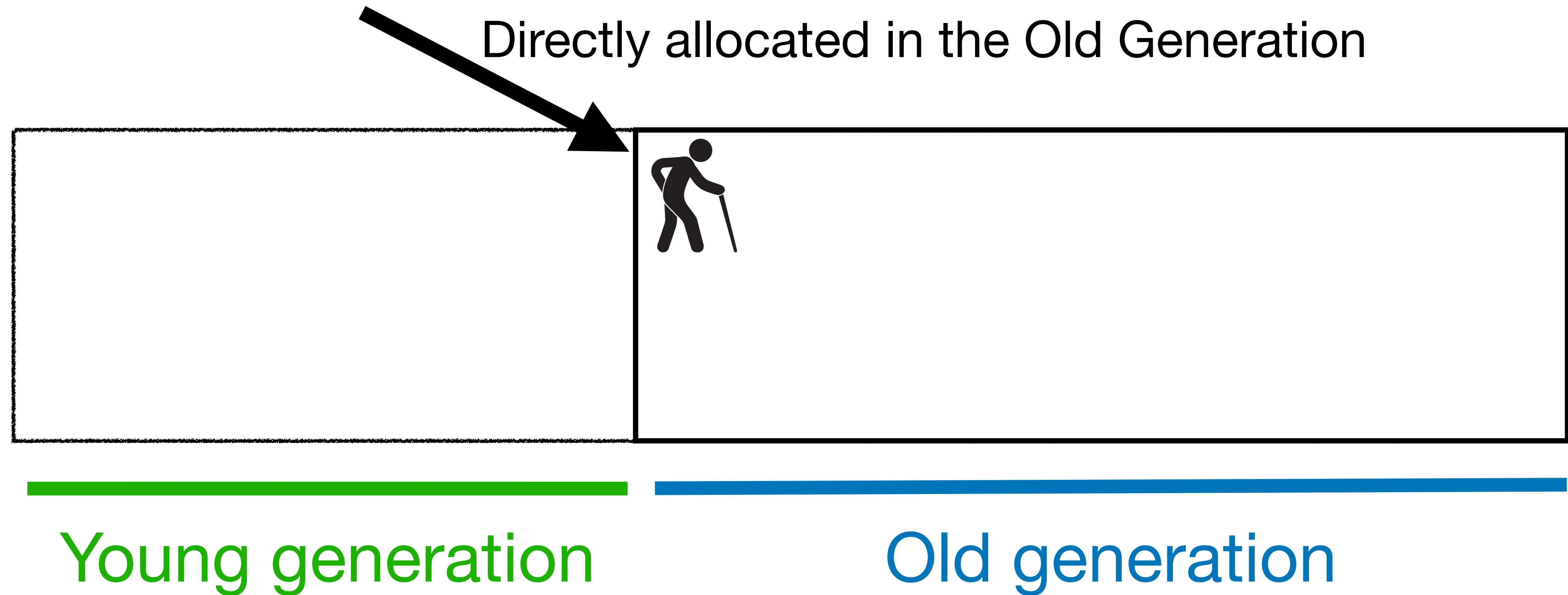


Enables



Pretenuring

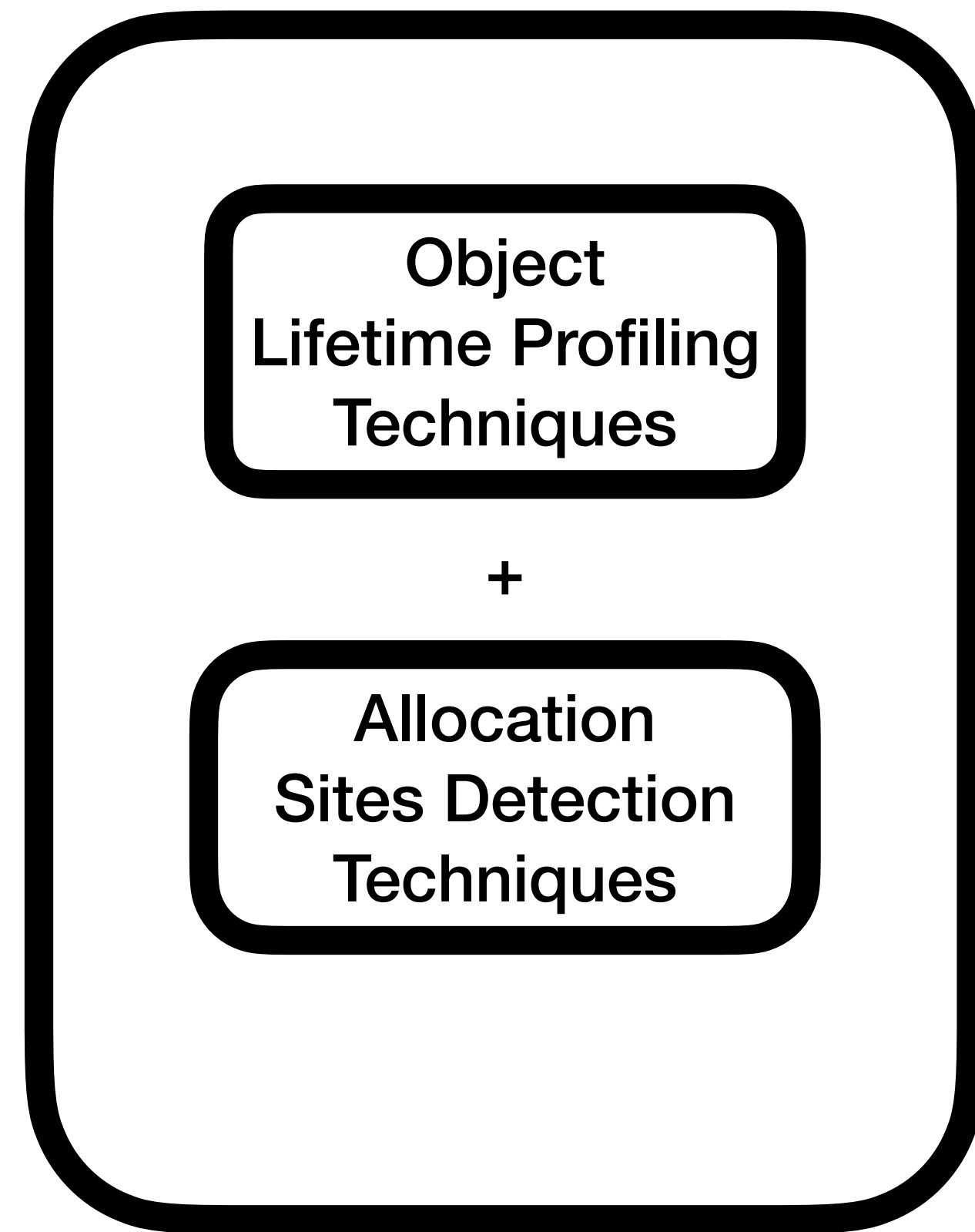
Pretenuring



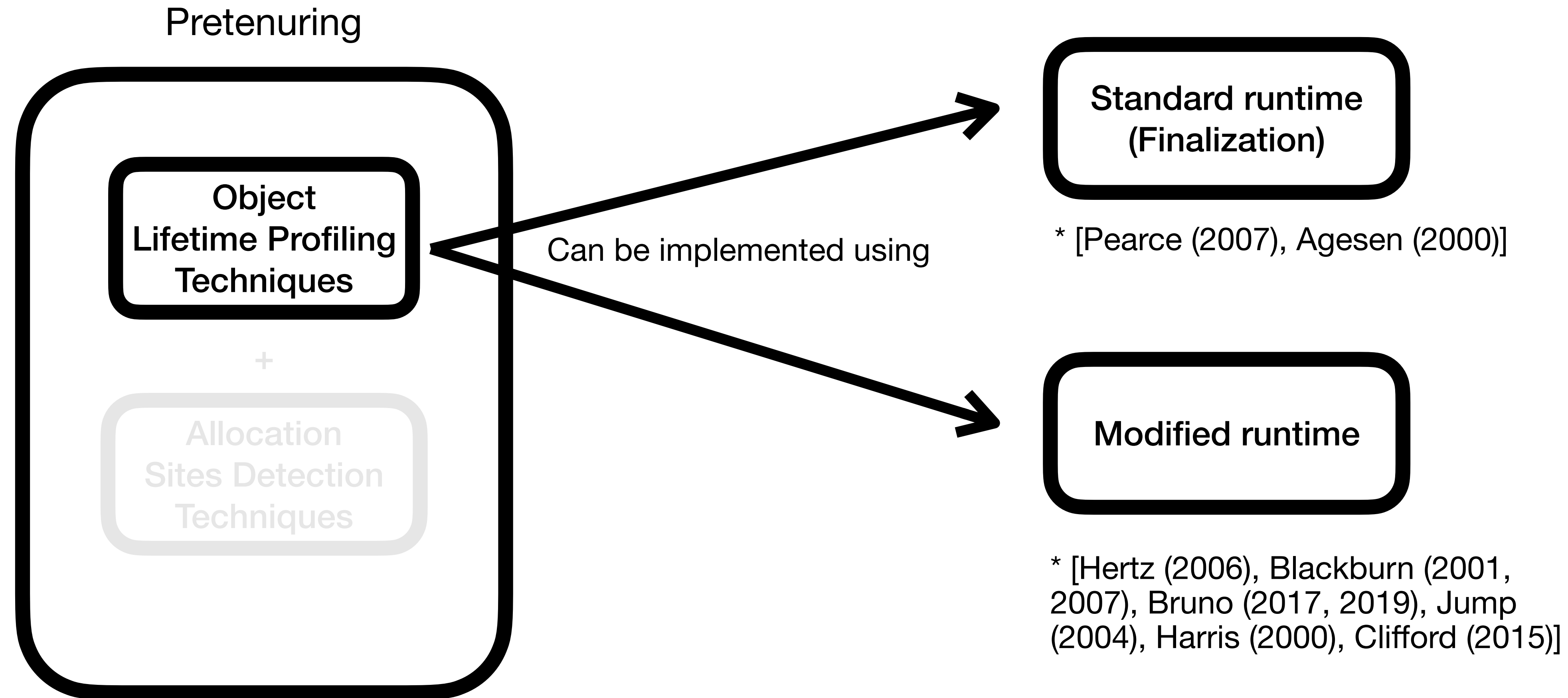
* [Harris (2000), Blackburn (2001, 2007), Jump (2004), Singer (2007), Clifford (2015)]

Pretenuring State of the Art

Pretenuring



Pretenuring State of the Art



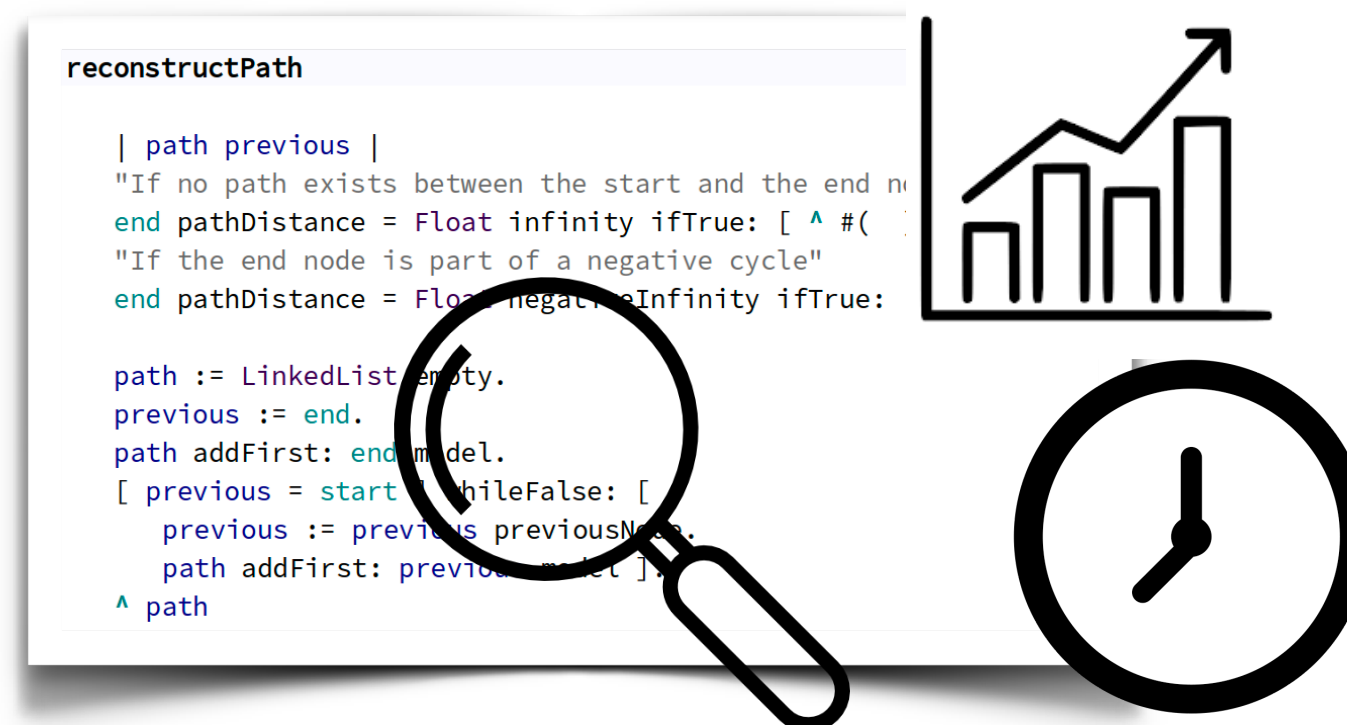
Object Lifetime Profiling: State of the Art

	Finalization-Based	VM Modification-Based
Development Cost	Low - Attach finalizers to sampled allocations.	High - Write barriers + root scans (Merlin); header bits (ROLP, Memento)
Portability	Yes - Easily deployable; Portability to other technologies	No - It depends on the runtime modifications
Overhead	Agesen no numbers; Pearce $\approx 1.26\times$ worst	70–300 $\times$ (Merlin); 1.05x-1.11x (ROLP)
Evaluation	Agesen: design-only proposal; Pearce: 10 benchmarks	Extensive evaluation

- **Finalization:** cheap to build, under-studied
- **VM modification:** expensive to build, well evidenced

We Focus on Finalization

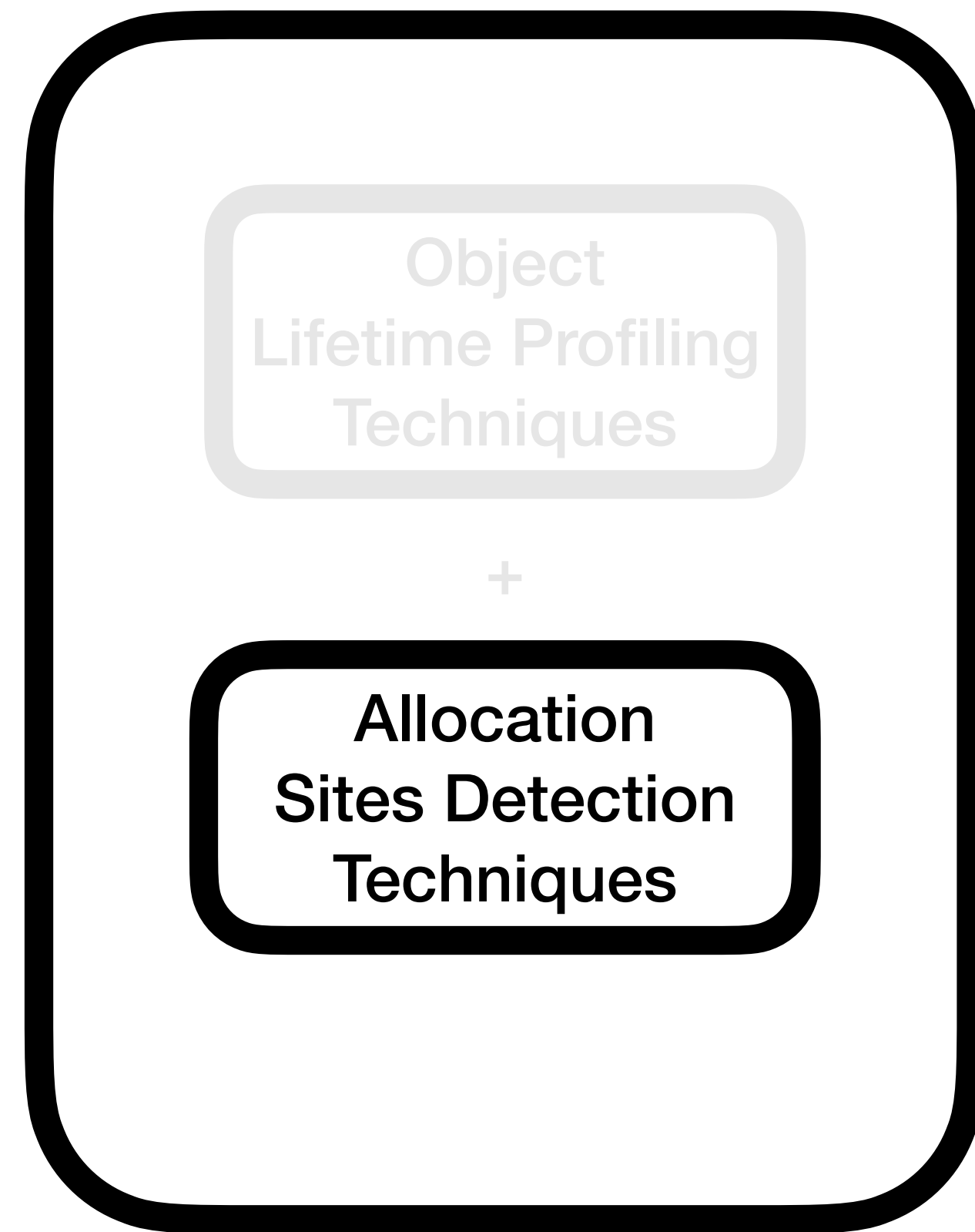
We want a **portable** solution that devs can load as a normal library.



Finalization-Based Lifetime Profiler

Pretenuring State of the Art

Pretenuring

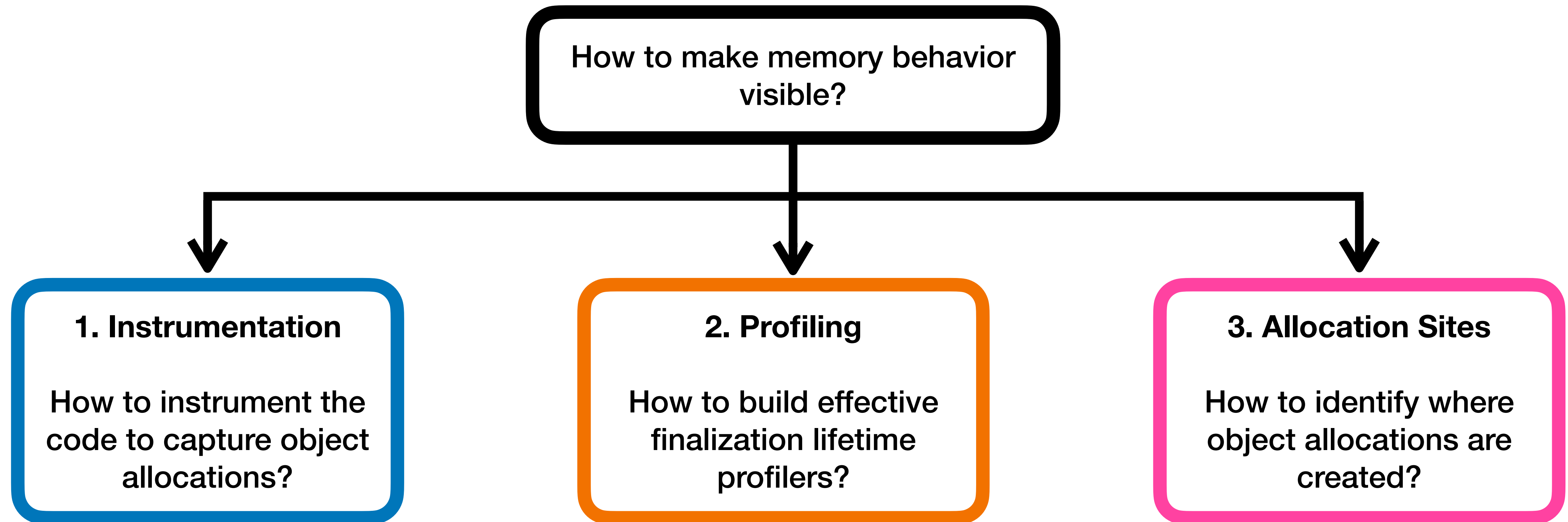


Allocation Site Identification: State of the Art

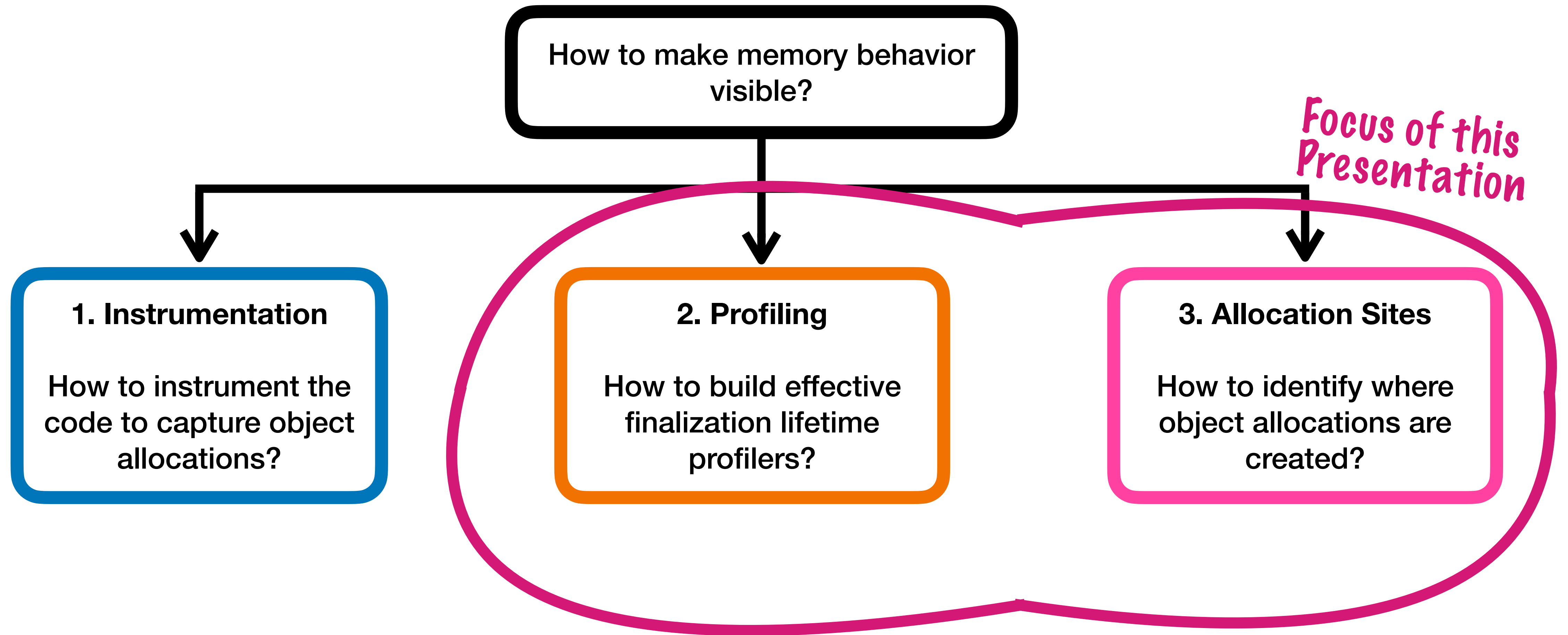
	Allocation Site Alone	Allocation Site + Calling Context
Information Used	Site only (lexical location). [Blackburn'01/'07, Bruno'17]	Site + call stack. [Singer'07, Inoue'06, Odaira'10, Bruno'19]
How Identified	Offline - Profile then derive sites. [Blackburn'01/'07, Bruno'17]	Online - Modify the object's header. [Odaira'10, Bruno'19, Clifford'15]
Tenuring Correlation	Weak - Same sites, different lifetimes. [Singer'07, Inoue'06]	Strong - Best predictor of lifetime. [Singer'07, Inoue'06]

- Allocation sites are shared by many callers
- Lifetime of the object depends on who uses it

Problem Statement



Problem Statement



Outline

1. Introduction
2. State of the Art
- 3. Finalization-Based Lifetime Profiling**
4. Textual Allocation Site Pretenuring
5. Path-Sensitive Pretenuring
6. Conclusion



2. Profiling

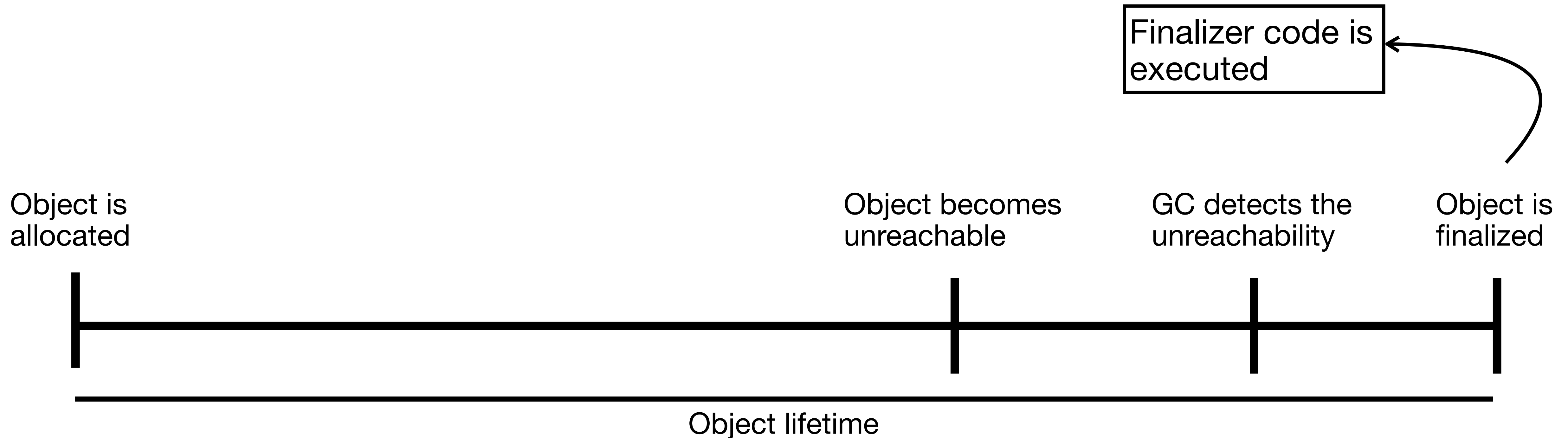
How to build effective
finalization lifetime
profilers?

What is Finalization?

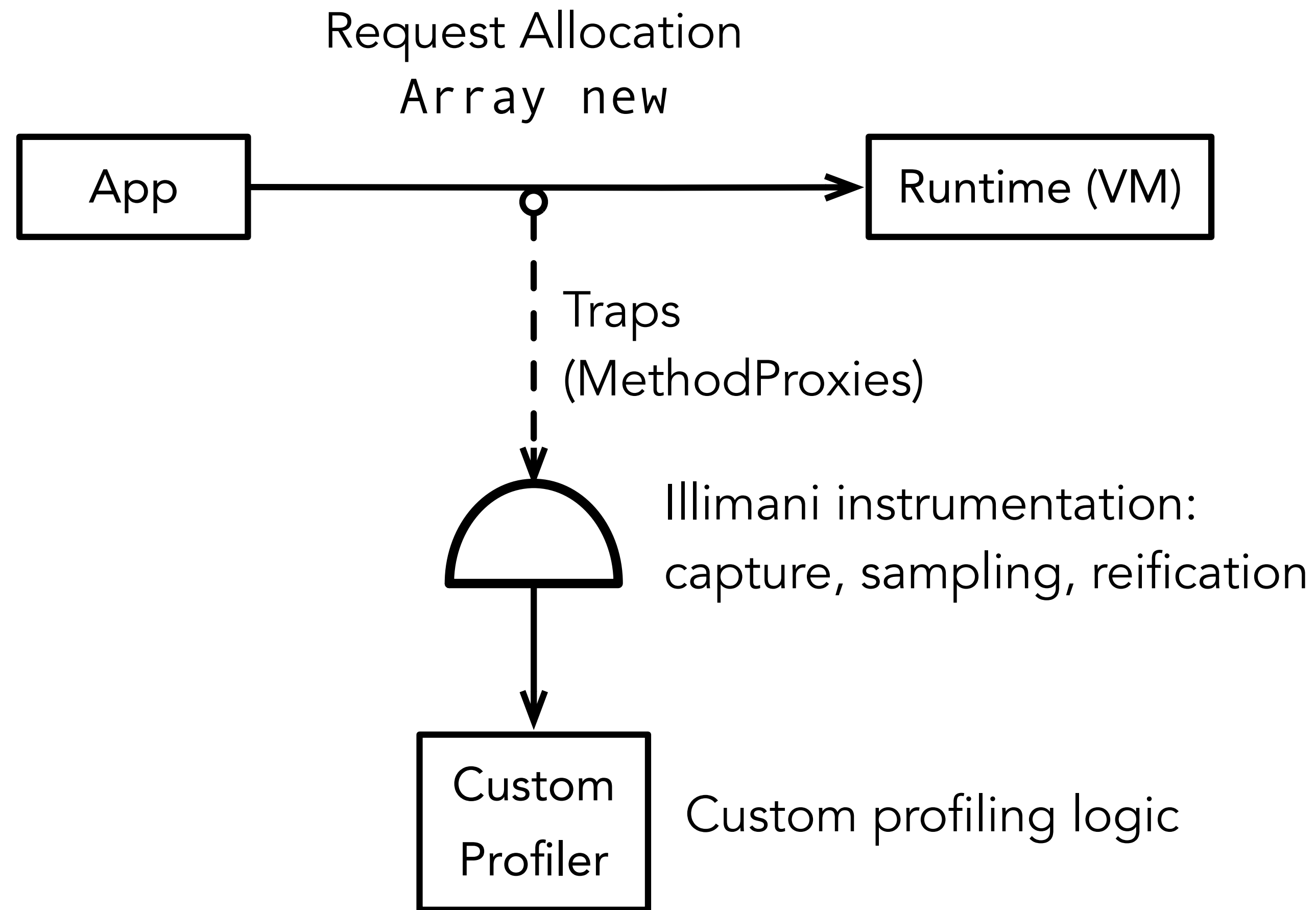
A hook to run code when an object becomes unreachable

What is Finalization?

A hook to run code when an object becomes unreachable



The Illimani Framework

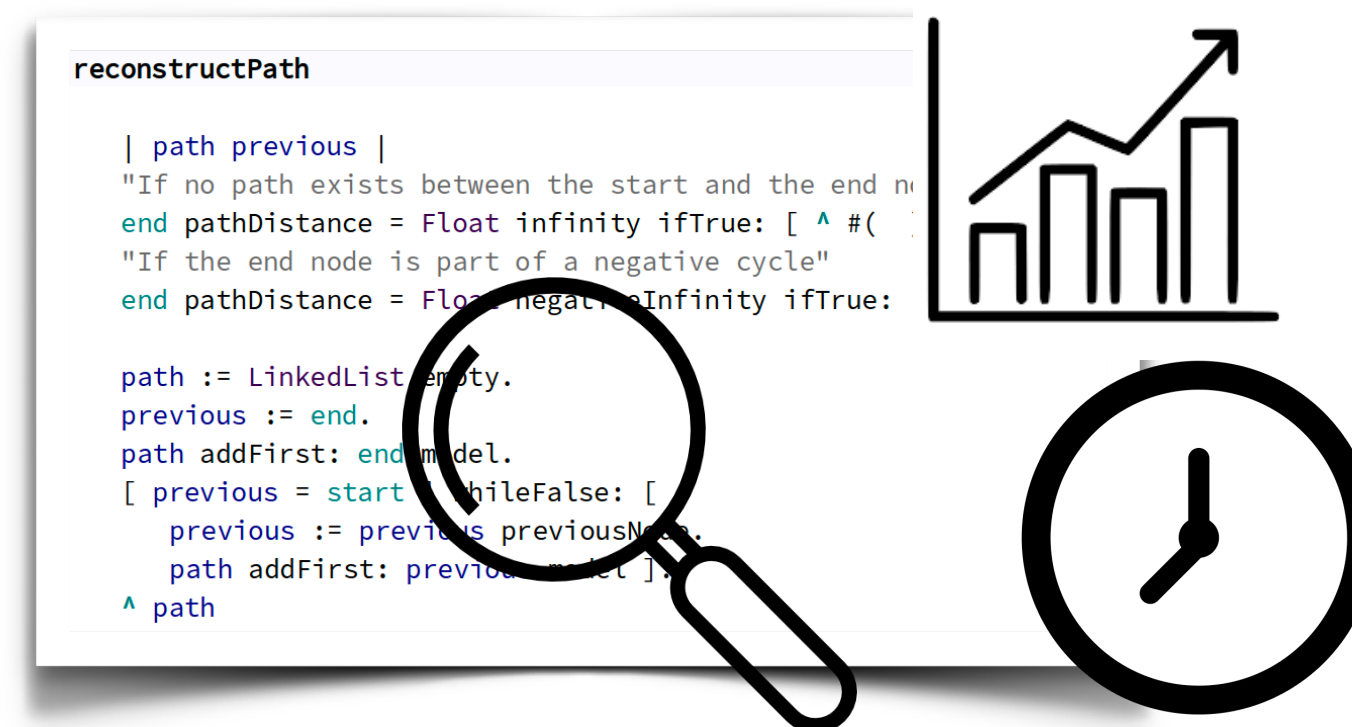
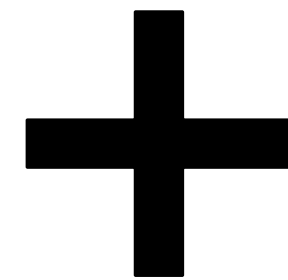


FiLiP

Finalization-Based Lifetime Profiler

Illimani

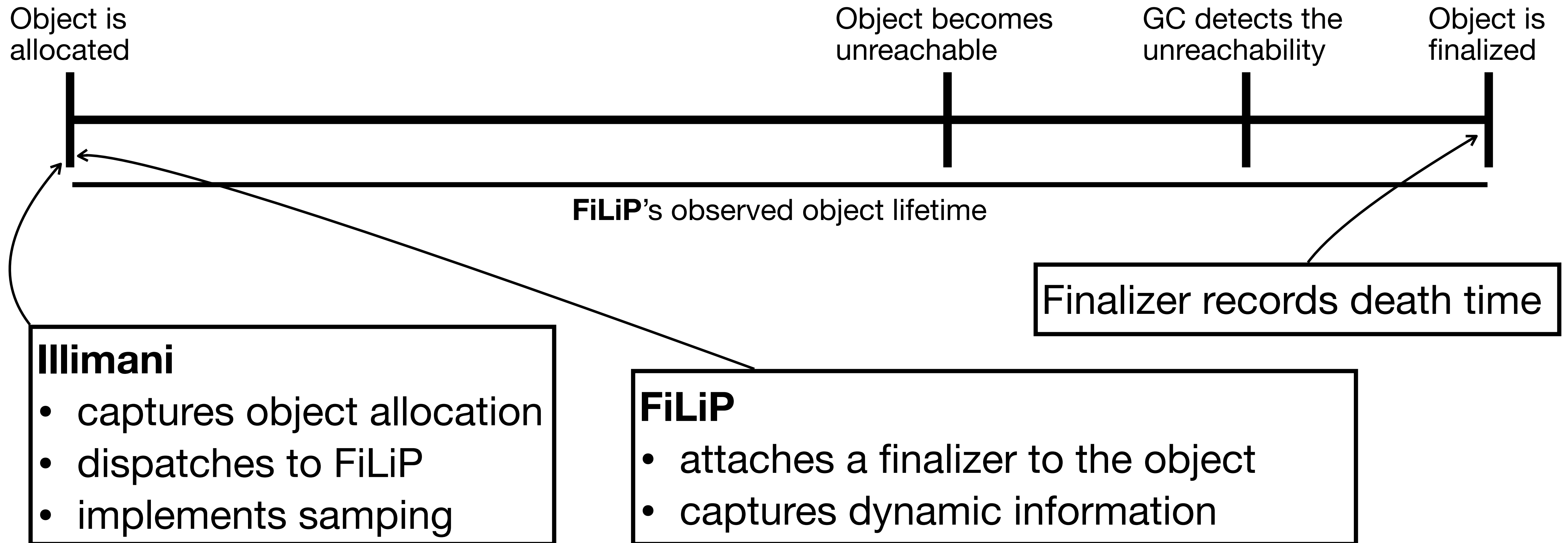
Framework for building
memory profilers



Finalization-Based Lifetime Profiler

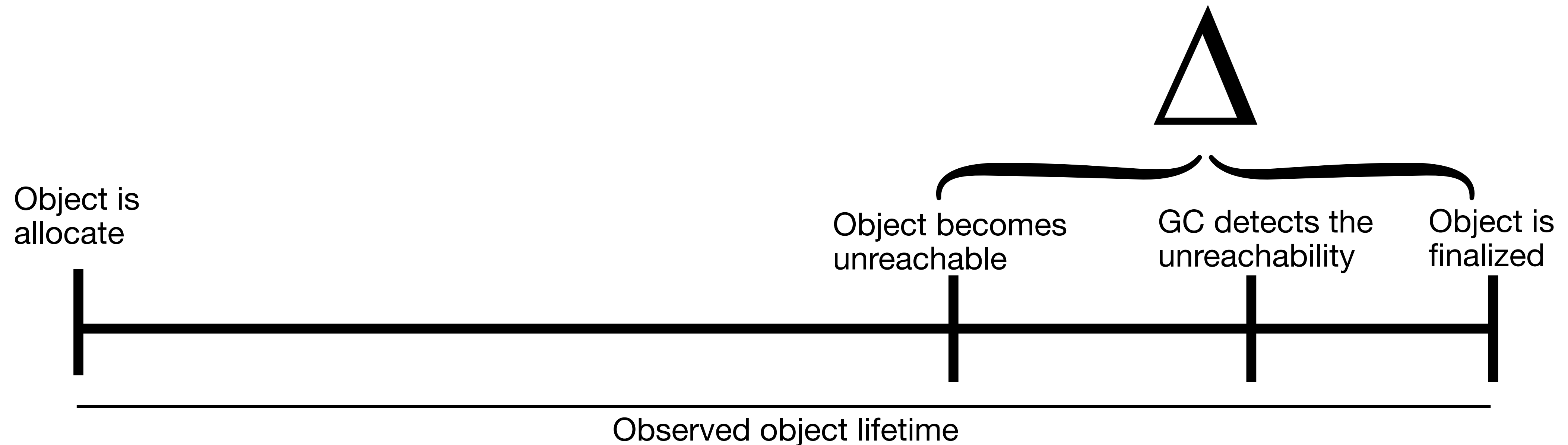
FiLiP in a Nutshell

Finalization-Based Lifetime Profiler



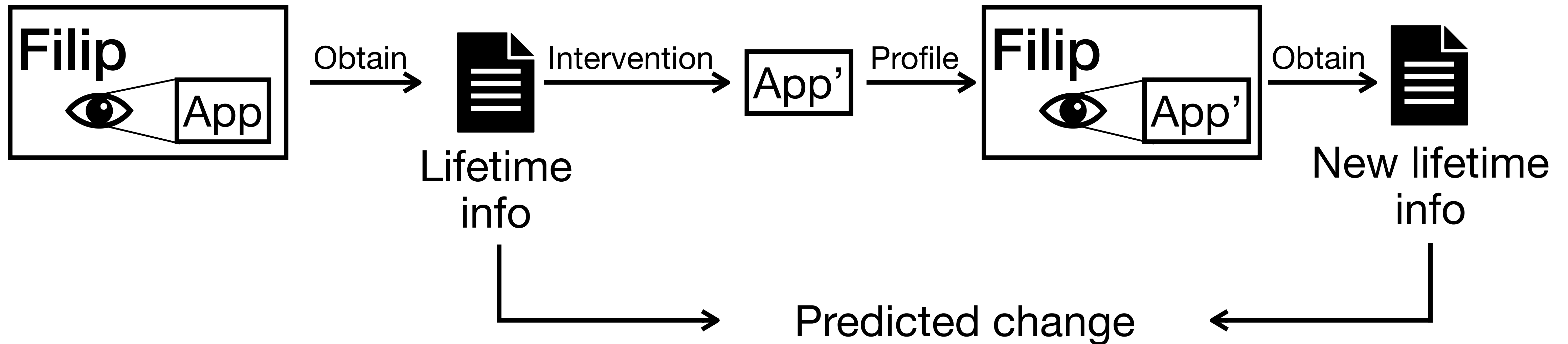
Observed Object Lifetime

Does the difference have an impact?
How can we evaluate it?



FiLiP's Actionability Methodology

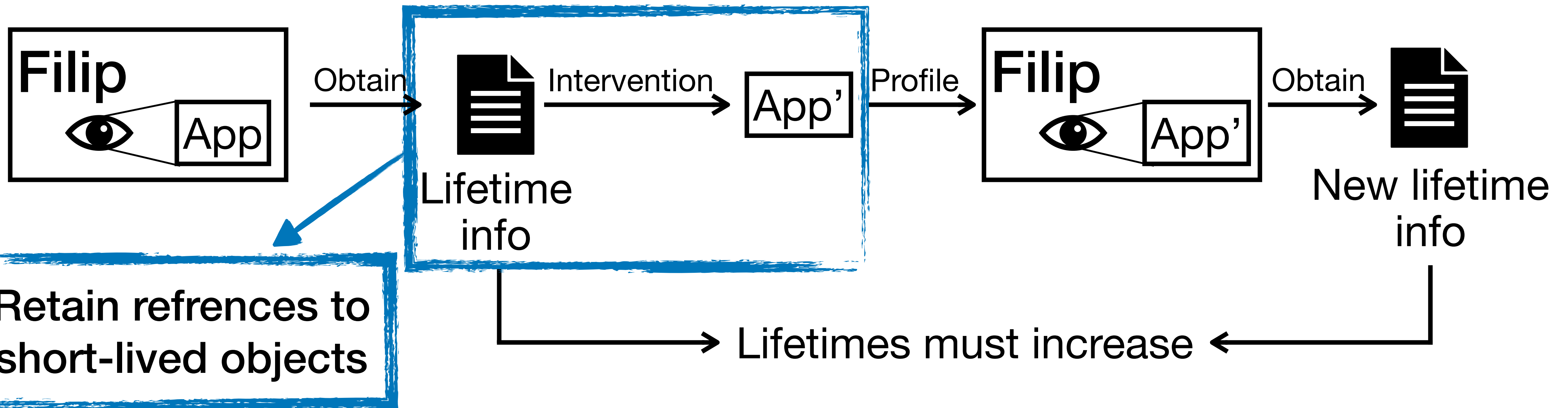
Methodology: Mytkowicz et al.'s causality analysis. [Mytkowicz '10]



Predicted change observed $\Rightarrow$ actionable

FiLiP's Actionability Methodology

Methodology: Mytkowicz et al.'s causality analysis. [Mytkowicz '10]



Predicted change observed $\Rightarrow$ actionable

Prediction: Lifetimes must go UP

FiLiP's Research Questions

- **RQ.1 Actionability.** Can we act on FiLiP's samples?
- **RQ.2 Object Lifetimes Precision.** How do different sampling rates impact the computed object lifetimes?
- **RQ.3 Execution Time Overhead.** Does sampling object allocations reduce the overhead?
- **RQ.4 Memory Overhead.** How much memory overhead does FiLiP introduce?

FiLiP's Research Questions

*Focus of this
Presentation*

- **RQ.1 Actionability.** Can we act on FiLiP's samples?
- **RQ.2 Object Lifetimes Precision.** How do different sampling rates impact the computed object lifetimes?
- **RQ.3 Execution Time Overhead.** Does sampling object allocations reduce the overhead?
- **RQ.4 Memory Overhead.** How much memory overhead does FiLiP introduce?

FiLiP's Benchmark Suite

1. **DataFrame.** Loaded a 230 MB dataset with 2,000,000 rows and 6 columns
2. **HoneyGinger.** Rendered a physics simulation
3. **Re:Mobidyc.** Agent-based simulation where wolves chase and eat goats.
4. **Bloc.** UI low-level library. Rendered the Boids simulation.
5. **Moose.** Loaded a bioinformatics project into the Moose meta model.

FiLiP RQ.1 Actionability

+ Positive means actionability

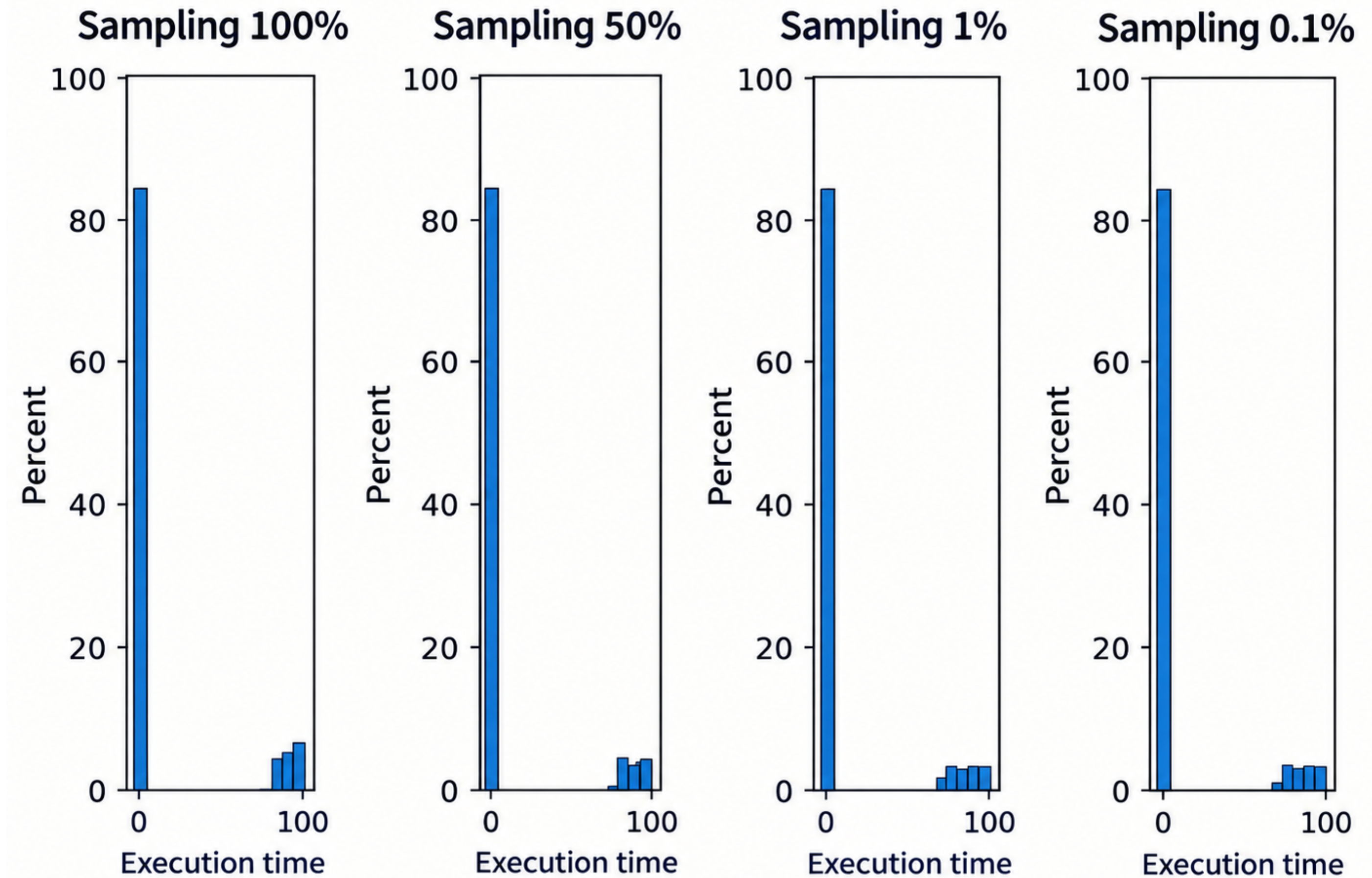
Application	Sampling rates (%)			
	100%	50%	1%	0.1%
DataFrame	+60%	+42%	+32%	+33%
HoneyGinger	+76%	+56%	+47%	+47%
Re:Mobidyc	+34%	+32%	+24%	+23%
Bloc	+25%	+13%	+13%	+14%
Moose	+10%	+9%	+10%	+10%

- Significant increases are observed **across all sampling rates**.
- **Anchoring effect:** the lifetimes of all the object graph references also increased.
- FiLiP produces **actionable object lifetimes**

FiLiP RQ. 2 Precision Across Sampling Rates

Lifetime frequencies for baseline profiler

- **Similar proportion:** ~85% short-lived; ~15% long-lived
- Results are **stable across all sampling rates**



Lifetime frequency for DataFrame across sampling rates

FiLiP RQ. 3 Execution Time Overhead

↓ Lower is better

Application	100%	50%	1%	0.1%
DataFrame	2.29×	1.64×	1.08×	1.12×
HoneyGinger	3.20×	2.50×	2.20×	1.80×
Bloc	1.03×	1.03×	1.03×	1.03×
Moose	2.02×	1.60×	1.10×	1.09×
Re:Mobidyc	2.06×	1.50×	1.30×	1.25×
Average	2.12×	1.65×	1.34×	1.26×

FiLiP remains **practical to use** with an average overhead of 1.26x with a 0.1% sampling rate.

FiLiP RQ. 4 Memory Overhead

↓ Lower is better

	DataFrame	HoneyGinger	Bloc	Moose	Mobidyc
Total memory	5053 MB	3312 MB	9 MB	3081 MB	4906 MB
Memory Overhead					
100% sampling	2.36 ×	3.34 ×	2.9 ×	2.61 ×	2.53 ×
50% sampling	1.68 ×	2.25 ×	2.0 ×	1.81 ×	1.75 ×
1% sampling	1.01 ×	1.03 ×	1.02 ×	1.02 ×	1.01 ×
0.1% sampling	1.001 ×	1.003 ×	1.002 ×	1.002 ×	1.001 ×

At 0.1% sampling rate, memory overhead is **negligible** ($\approx 1.00\times$).

FiLiP: Use Cases

Four real-world memory issues identified, fixed and integrated:

- Redundant Color allocations in UI theme.

*Integrated into Pharo 11

- Excessive temporary collections
- Intermediate collections in hot execution path
- Excessive NumberParser instantiation

Independently identified

*Integrated into Moose

Outline

1. Introduction
2. State of the Art
3. Finalization-Based Lifetime Profiling
- 4. Textual Allocation Site Pretenuring**
5. Path-Sensitive Pretenuring (PSP)
6. Conclusion

2. Profiling

How to build effective finalization lifetime profilers?

3. Allocation Sites

How to identify where object allocations are created?

The Textual Allocation Site

Static lexical location: method and bytecode offset

```
public void method() {  
    new ArrayList<>();  
}
```

Java

```
method  
    Array new
```

Pharo

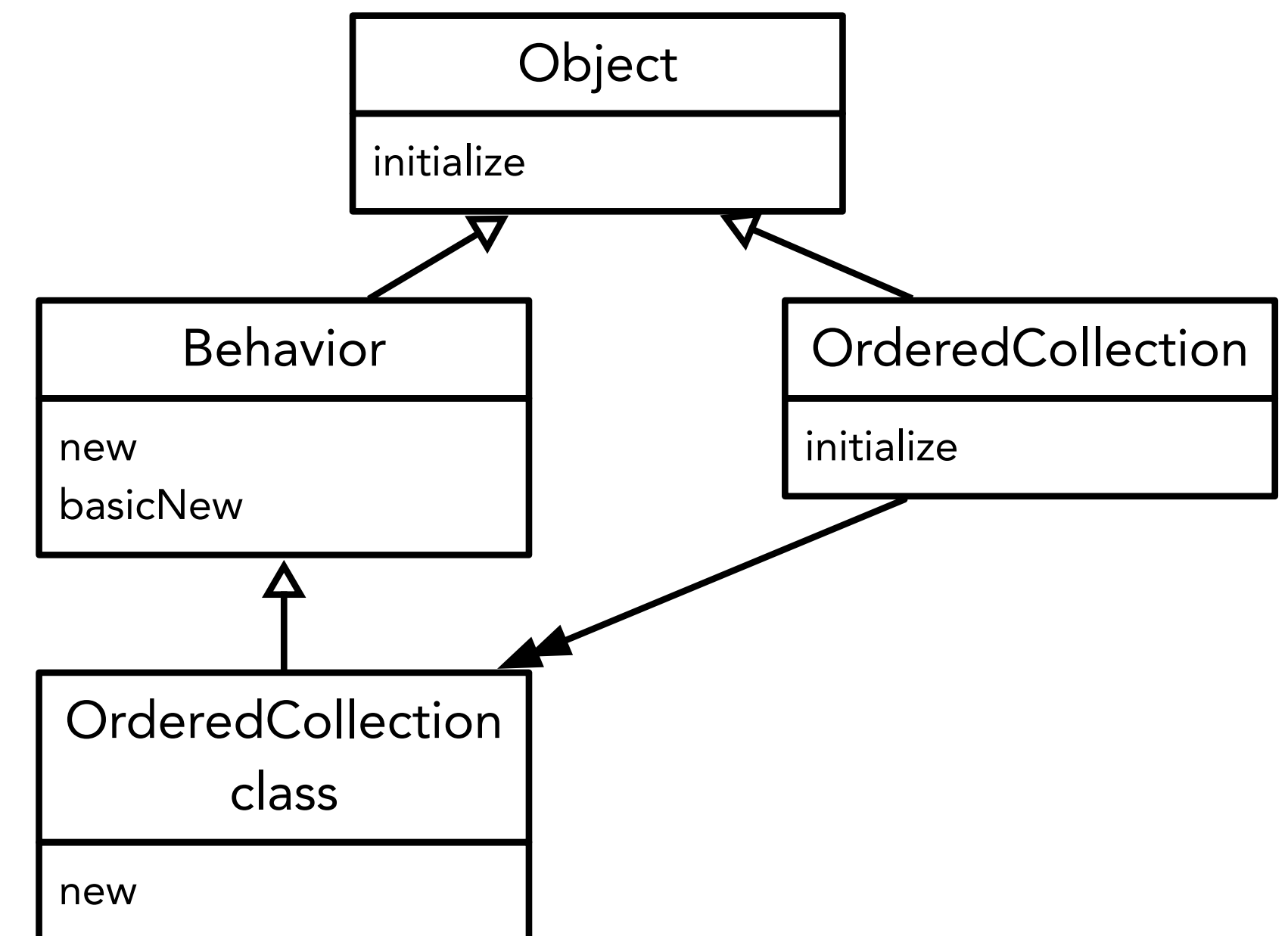
Redefining The Textual Allocation Site

In Java (static):

- new compiles to a special bytecode
- Each site is **unique**

In Pharo (dynamic):

- new is a regular method
- All allocations resolve to `basicNew`
- Textual sites **collapse to a few methods**
- *Traverse the call stack until finding the method that sends new*

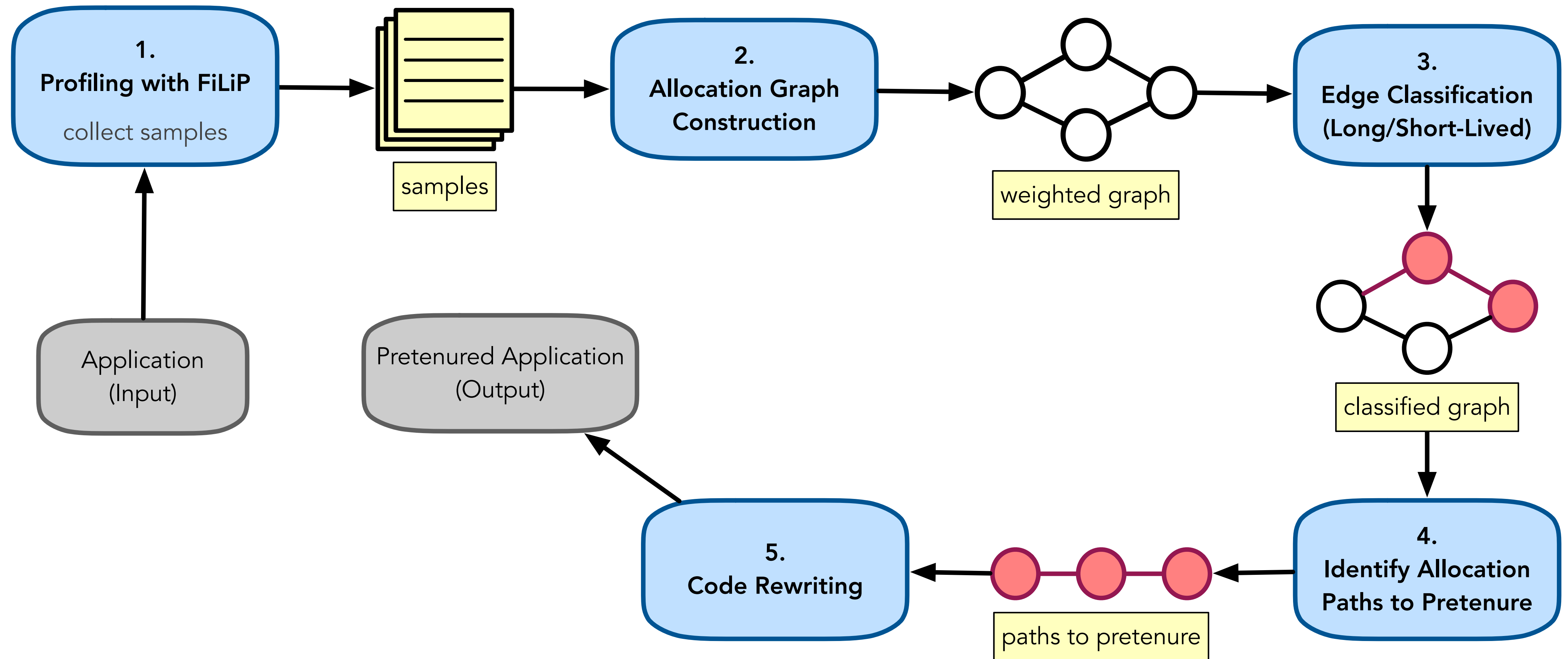


Legend

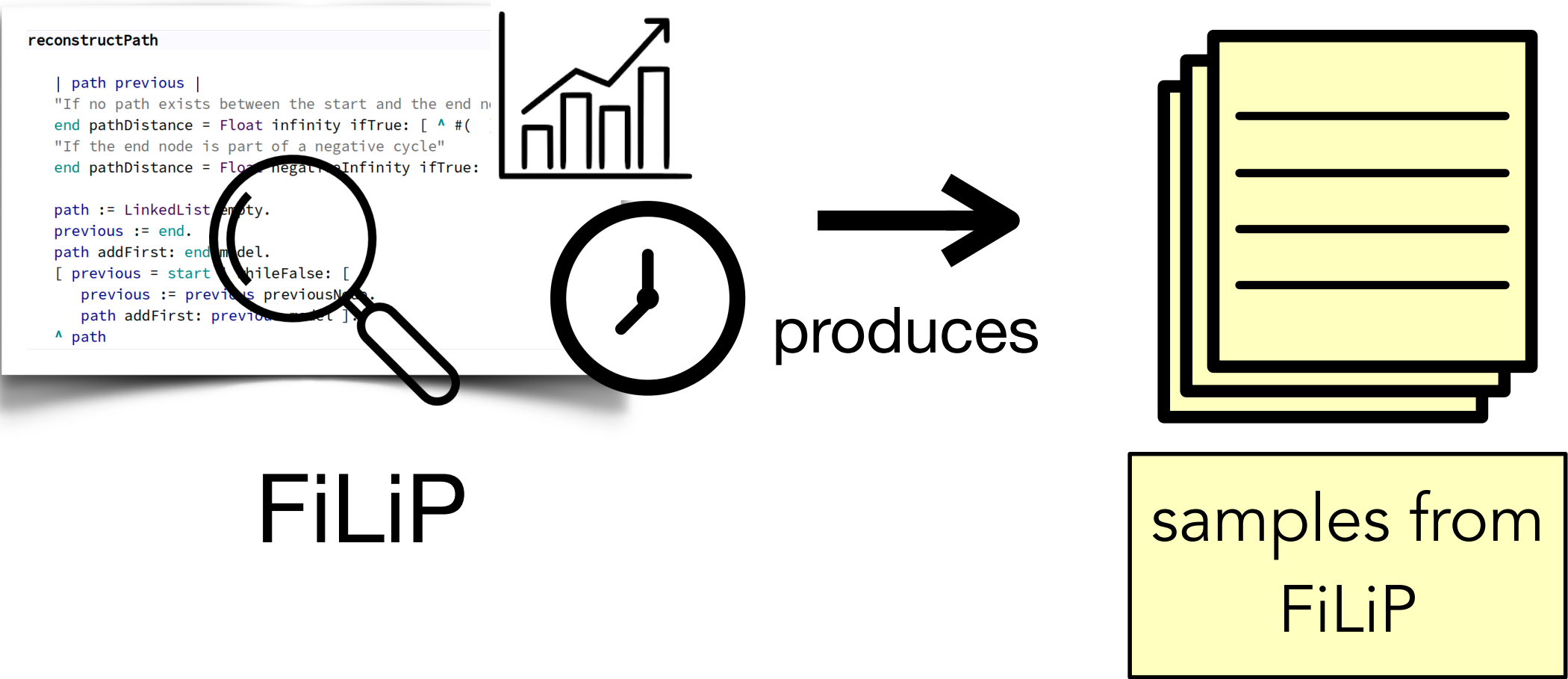
A $\twoheadrightarrow$ B B is the metaclass of A
A $\rightarrow$ B A inherits from B

Pharo's Simplified Meta Model

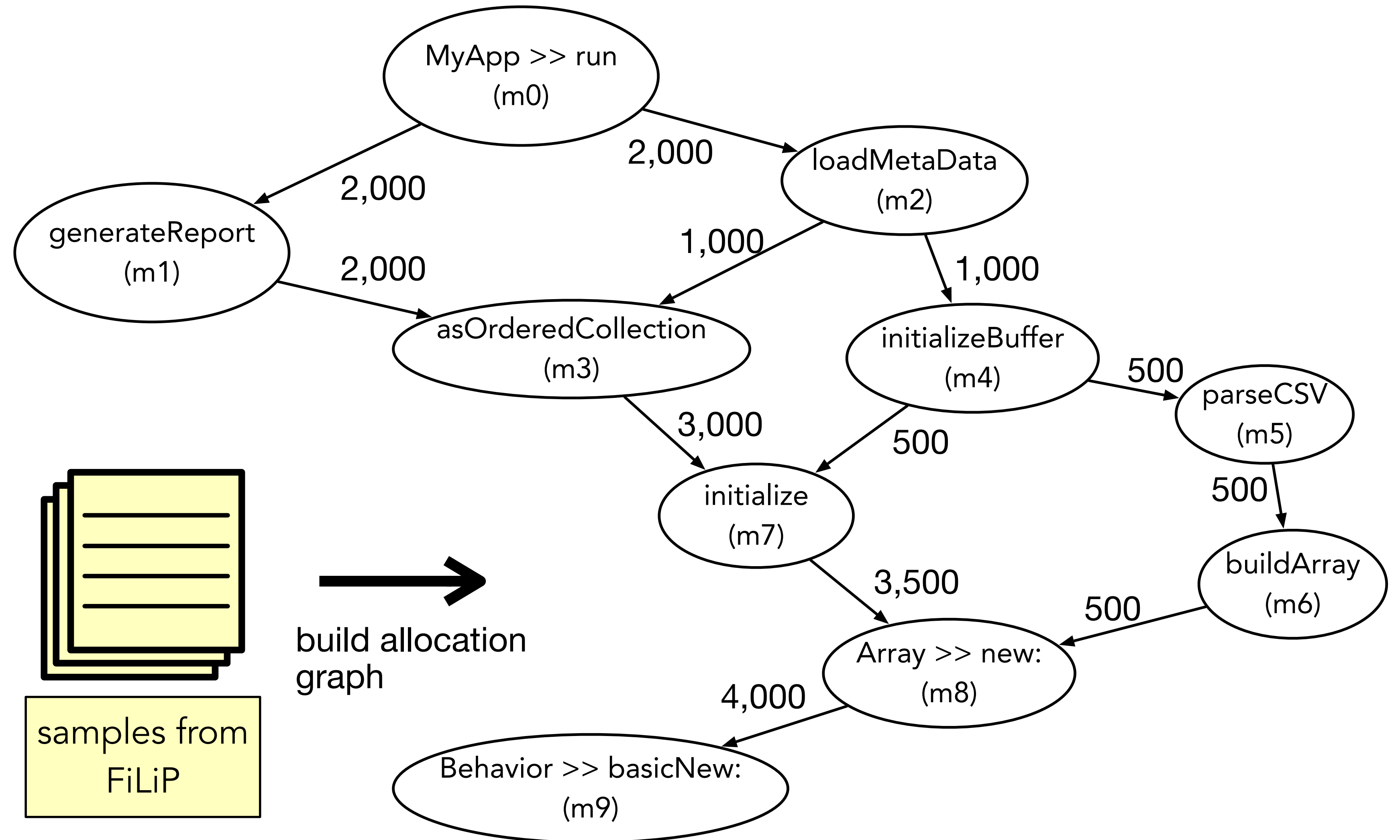
Pretenuring Algorithm Overview



1. Profiling the Application

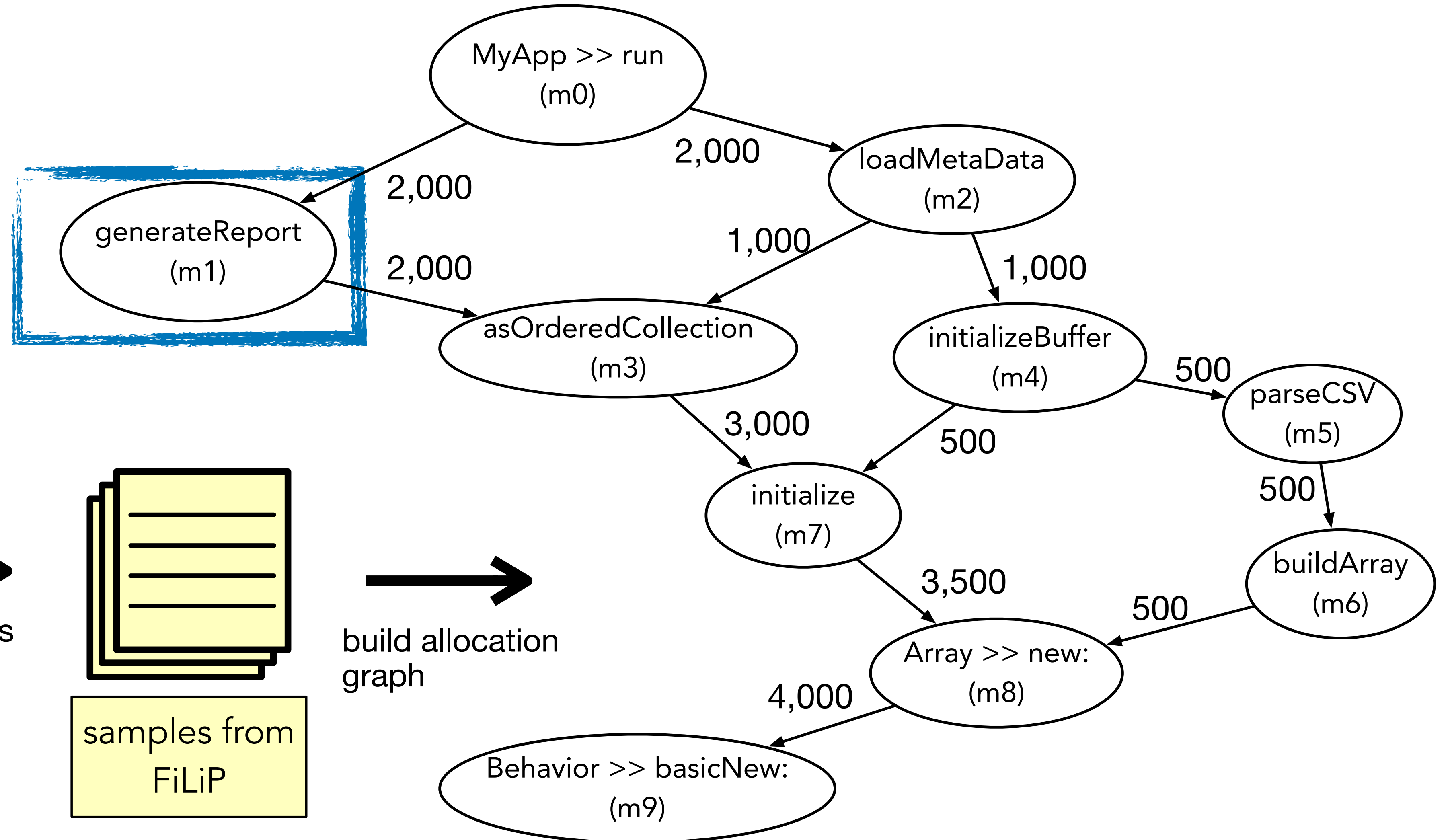


2. Allocation Graph Construction



2. Allocation Graph Construction

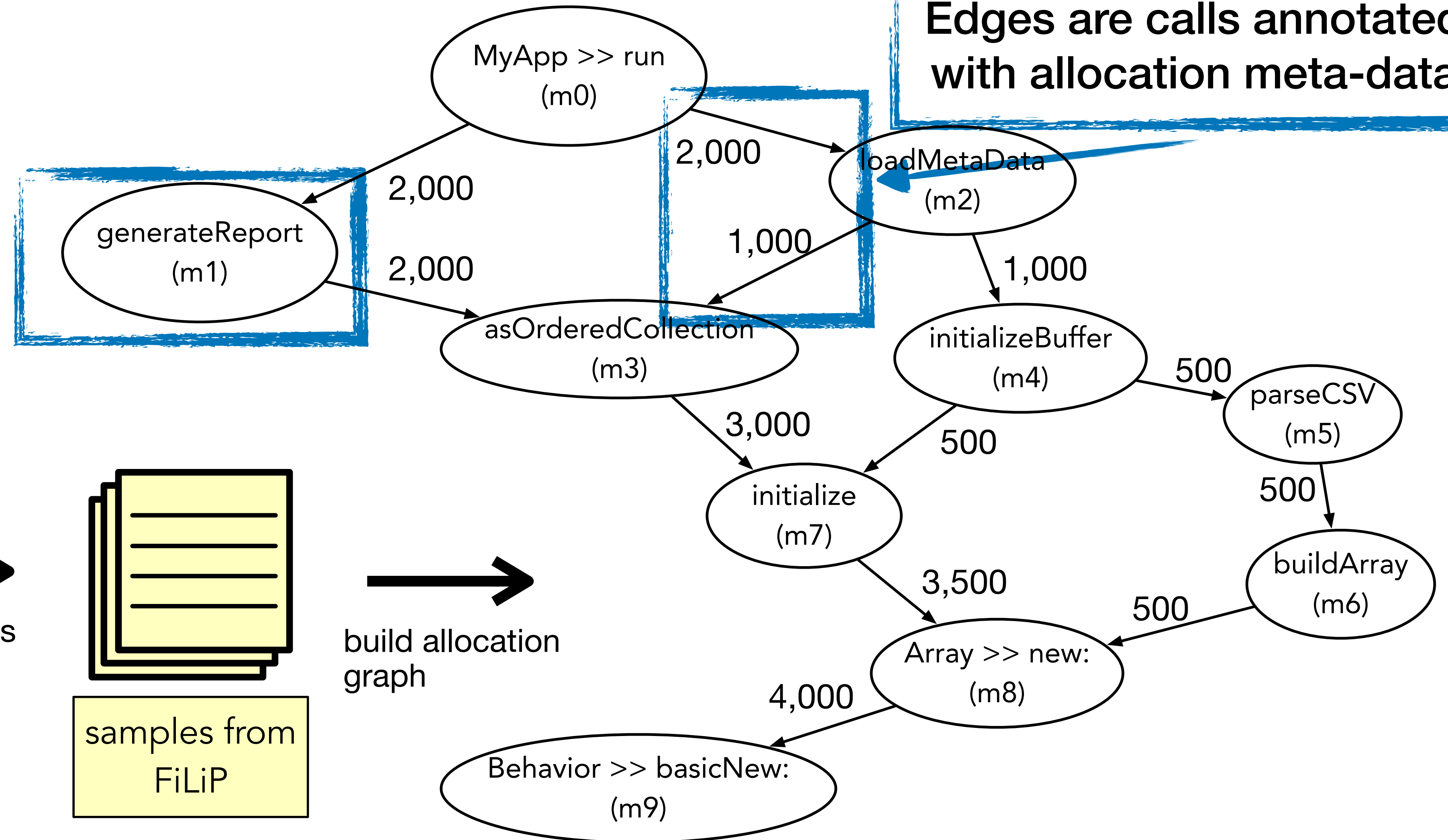
Nodes are methods



2. Allocation Graph Construction

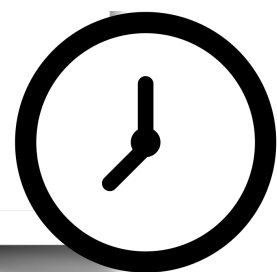
Nodes are methods

Edges are calls annotated with allocation meta-data



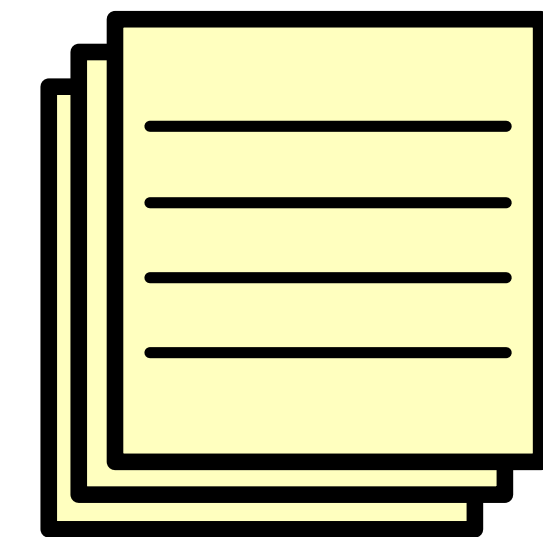
reconstructPath

```
| path previous |  
"If no path exists between the start and the end node"  
end pathDistance = Float infinity ifTrue: [ ^ #() ]  
"If the end node is part of a negative cycle"  
end pathDistance = Float infinity ifTrue: [ ^ #() ]  
  
path := LinkedList empty.  
previous := end.  
path addFirst: end model.  
[ previous = start ] whileFalse: [  
    previous := previous previous.  
    path addFirst: previous model.  
]  
^ path
```



produces

FiLiP



samples from
FiLiP

build allocation
graph

3. Classifying Edges

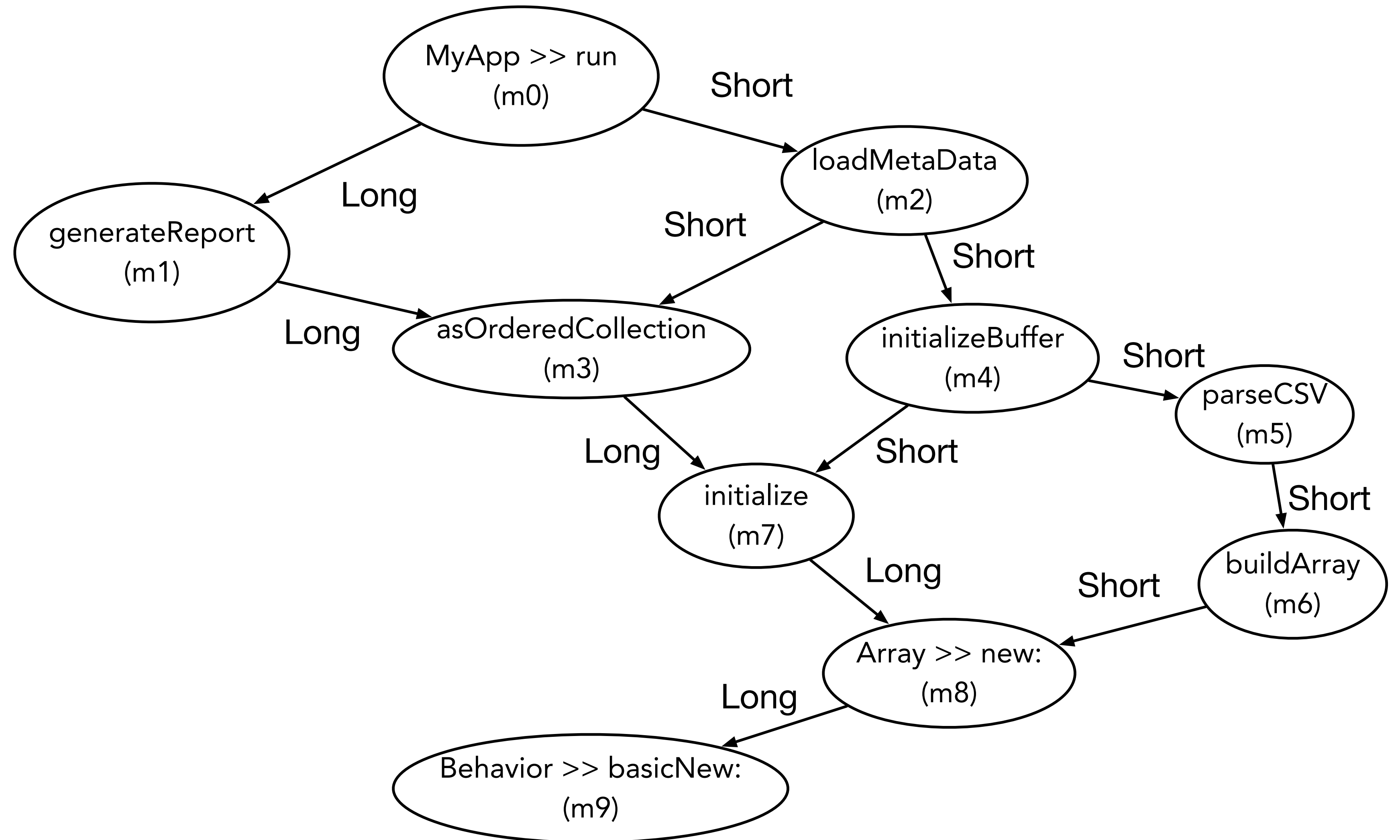
« **Pretenuring Advice** » algorithm, *Blackburn et al.* [Blackburn '07]

- Classifies edges and objects as *Short-lived*, *Long-lived* or *Immortal*

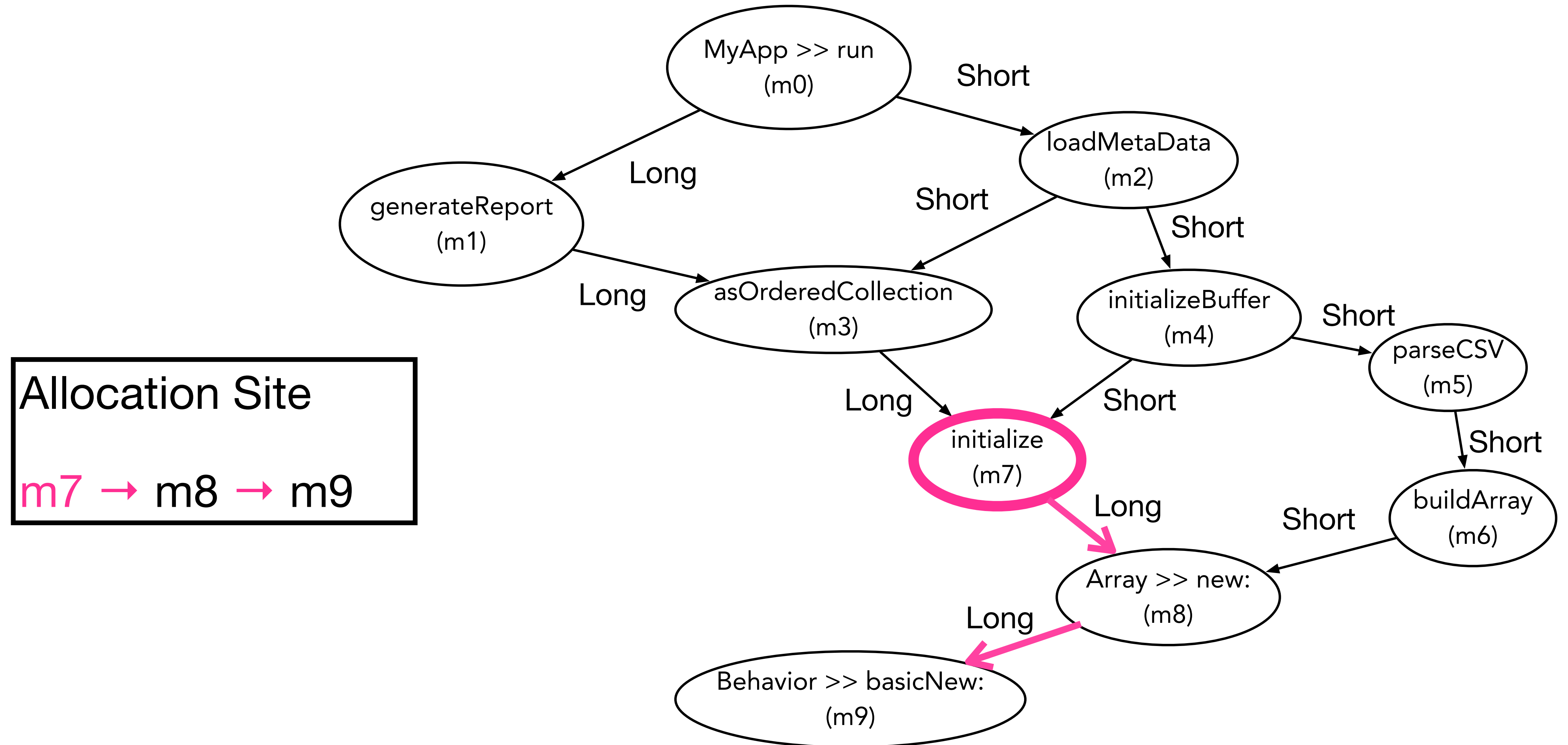
$$\text{isLong} = \text{long} > \text{short} + H$$

$H = 60\%$ of total allocations

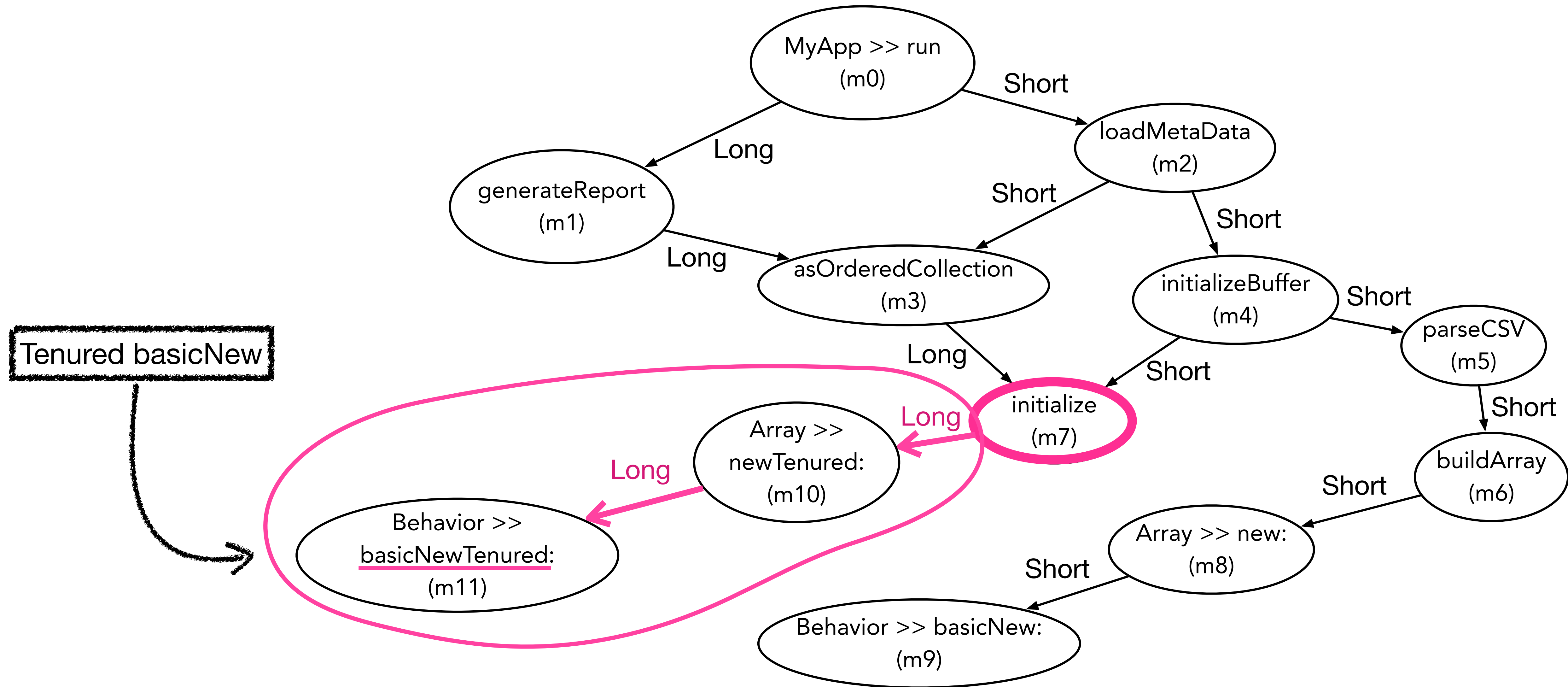
3. Classifying Edges



4. Graph Traversal



5. Code Rewriting



5. Code Rewriting

```
OrderedCollection >> initialize  
    super initialize.  
    internalArray := Array newTenured: 10.
```

```
Array >> newTenured: aSize  
    | instance |  
    instance := self basicNewTenured: aSize.  
    instance initialize.  
    ^ instance
```

Textual Allocation Site Research Questions

- **RQ.1 Performance.** Is the textual allocation site effective?
- **RQ.2 Garbage Collection Overhead.** How much GC overhead reduces?
- **RQ.3 JIT Compilation Overhead.** What is the impact on JIT compilation?

Textual Allocation Site Research Questions

*Focus of this
Presentation*

- **RQ.1 Performance.** Is the textual allocation site effective?
- **RQ.2 Garbage Collection Overhead.** How much GC overhead reduces?
- **RQ.3 JIT Compilation Overhead.** What is the impact on JIT compilation?

RQ. 1 Performance

Results:

- Results ranging from (0.96x–1.18x)
- High sampling rates **degrade performance**
- **No consistent benefit**

Sampling Rate	Cormas	DataFrame
0.1%	0.99	0.96
1%	0.97	0.96
10%	1.05	1.18
50%	1.05	1.16
100%	0.99	1.16

Textual allocation sites yield limited improvements with finalization-based profiling.

<1 is improvement; >1 is decrease

Outline

1. Introduction
2. State of the Art
3. Finalization-Based Lifetime Profiling
4. Textual Allocation Site Pretenuring
- 5. Path-Sensitive Pretenuring (PSP)**
6. Conclusion

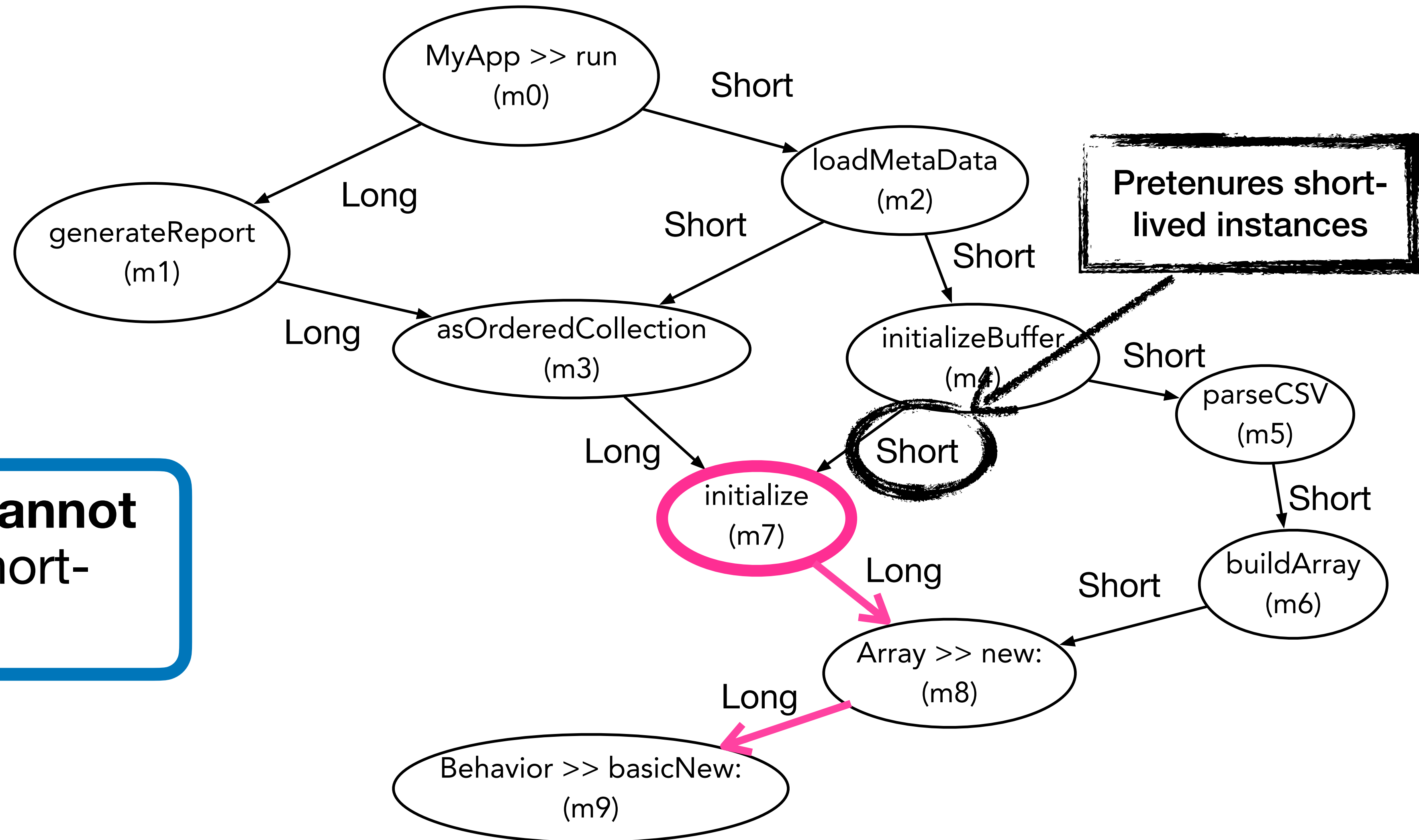
2. Profiling

How to build effective finalization lifetime profilers?

3. Allocation Sites

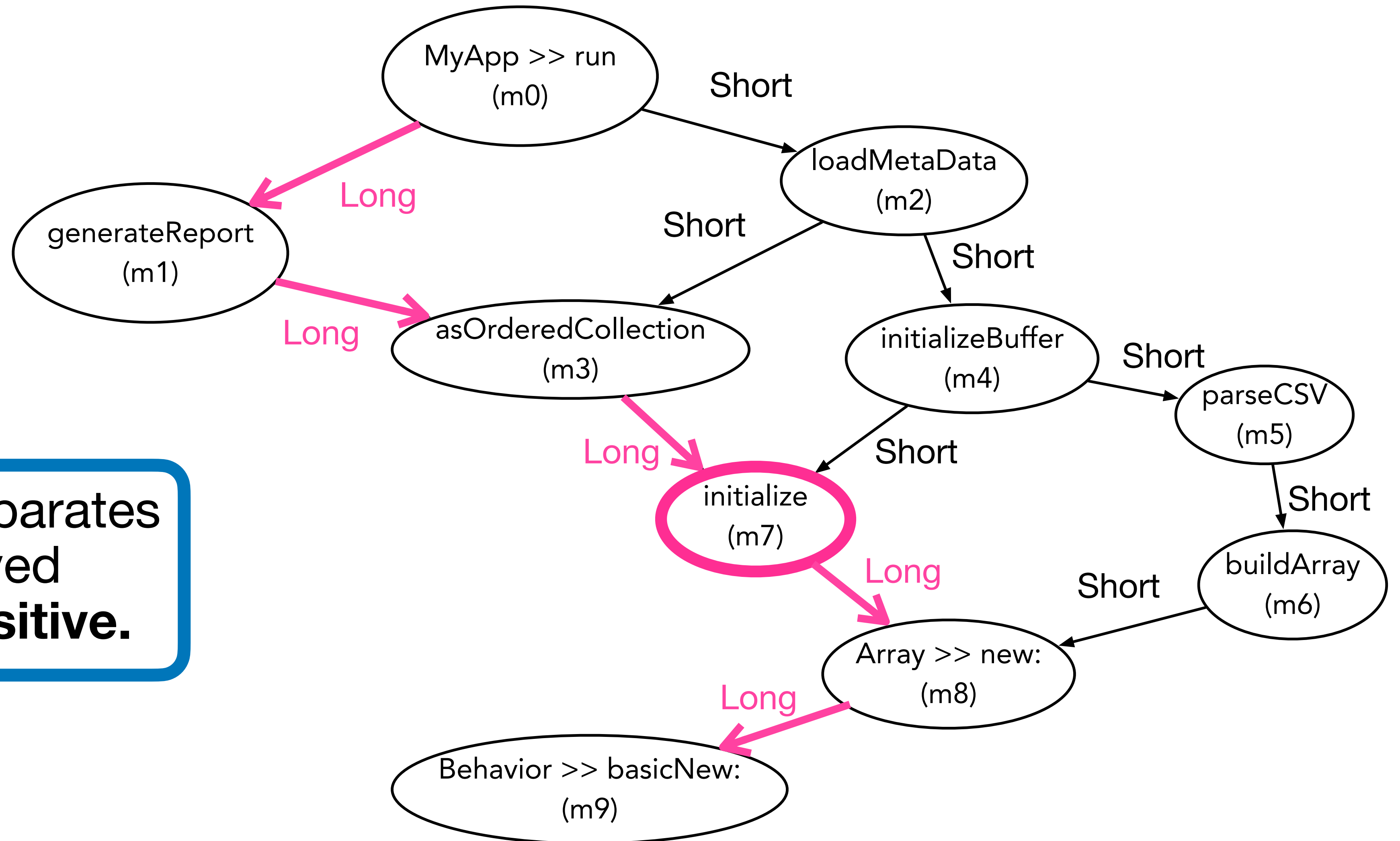
How to identify where object allocations are created?

The Textual Allocation Site Problem



Textual allocation site **cannot distinguish** between short- and long-lived objects

Path-Sensitive Pretenuring Motivation



The information that separates long-lived from short-lived allocations is **path-sensitive**.

Path-Sensitive Pretenuring

Extends the textual allocation site by adding **path-sensitive calling context information**.

Showdown: Textual vs. Path-Sensitive

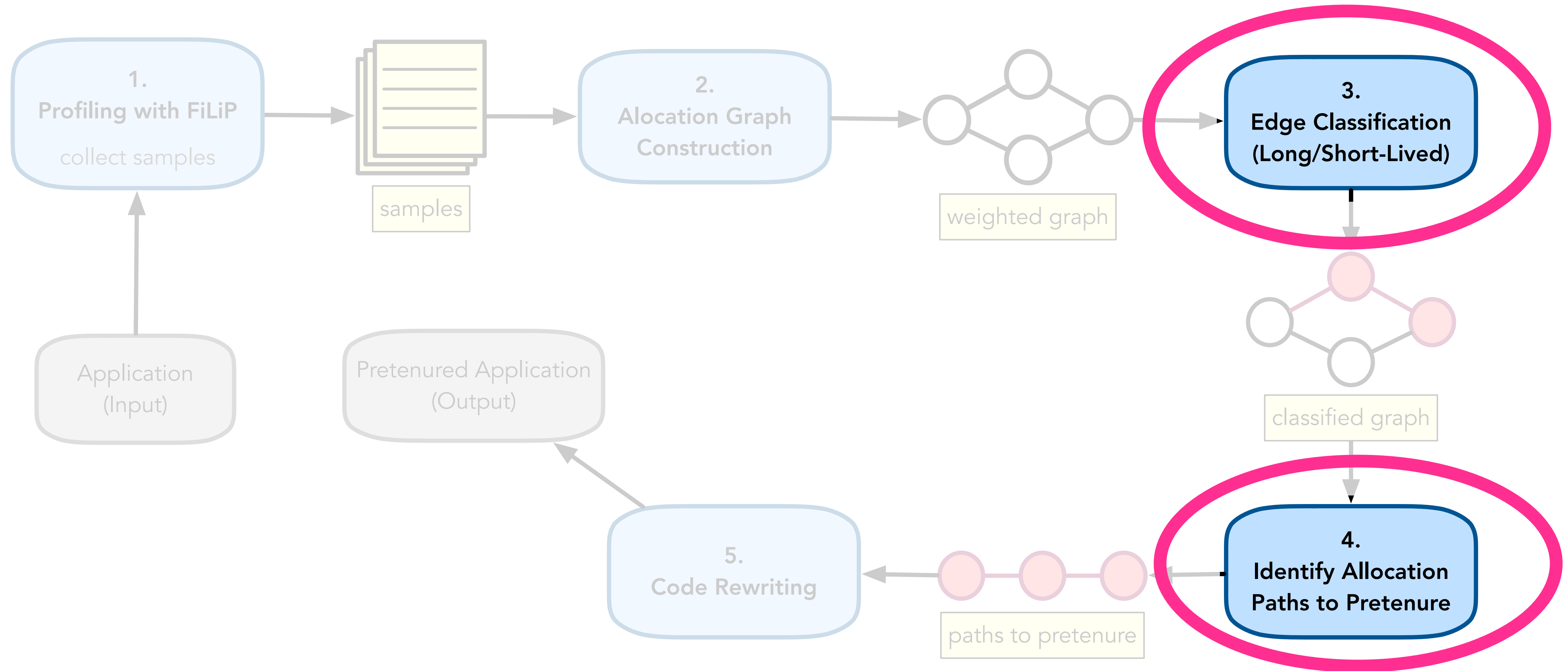
Textual site:

- Cannot distinguish long-lived from short-lived paths.
- Pretenures all allocations.

PSP:

- Pretenures only the long-lived path.
- Other callers remain unaffected.

PSP Algorithm Overview



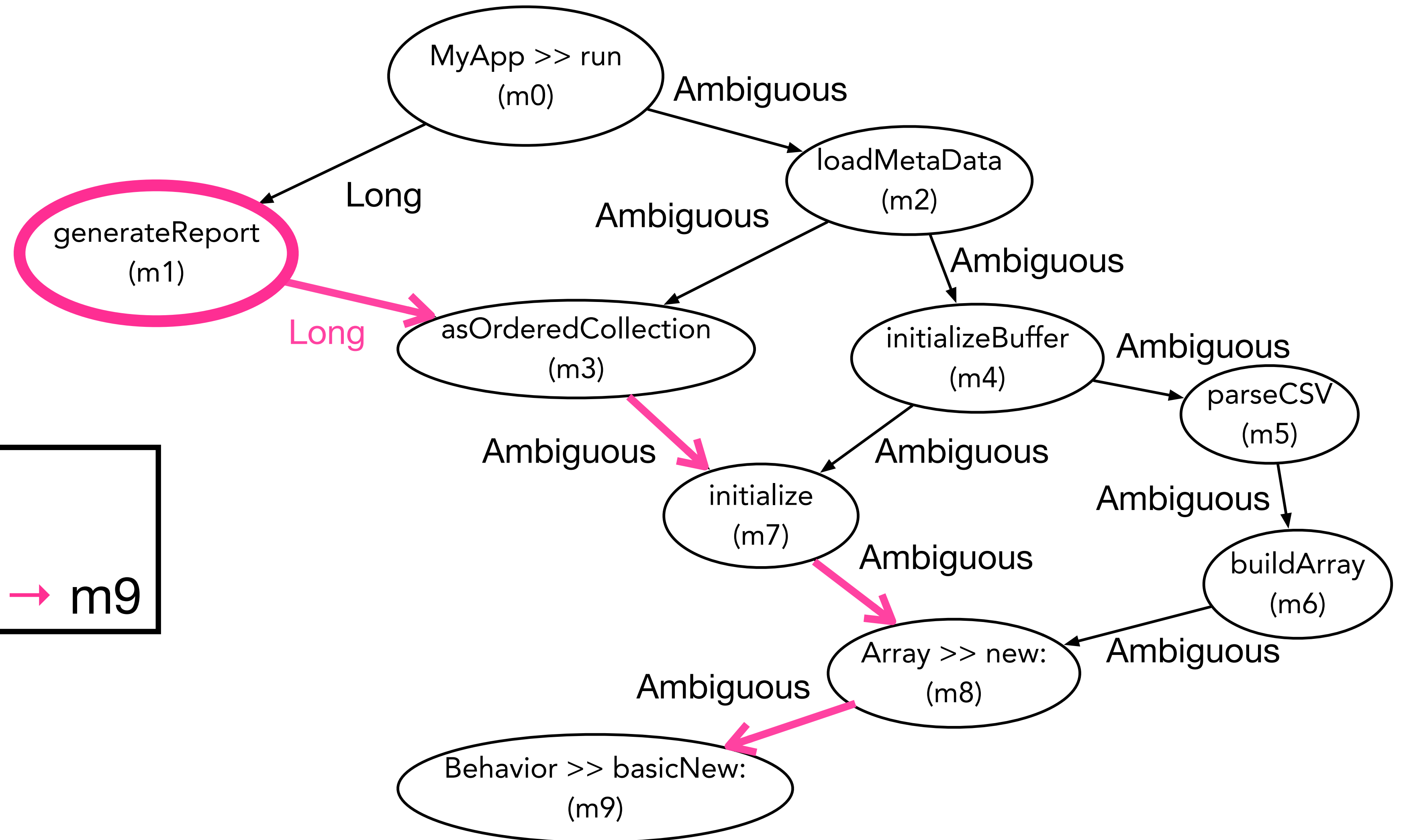
3. PSP Classification Algorithm

- Based on « Pretenuring Advice » by Blackburn *et. al*
- Two edge states: long-lived, or ***ambiguous***

`isLong = long > (totalAllocations * AT)`

AT = ambiguity threshold (e.g., 0.7)

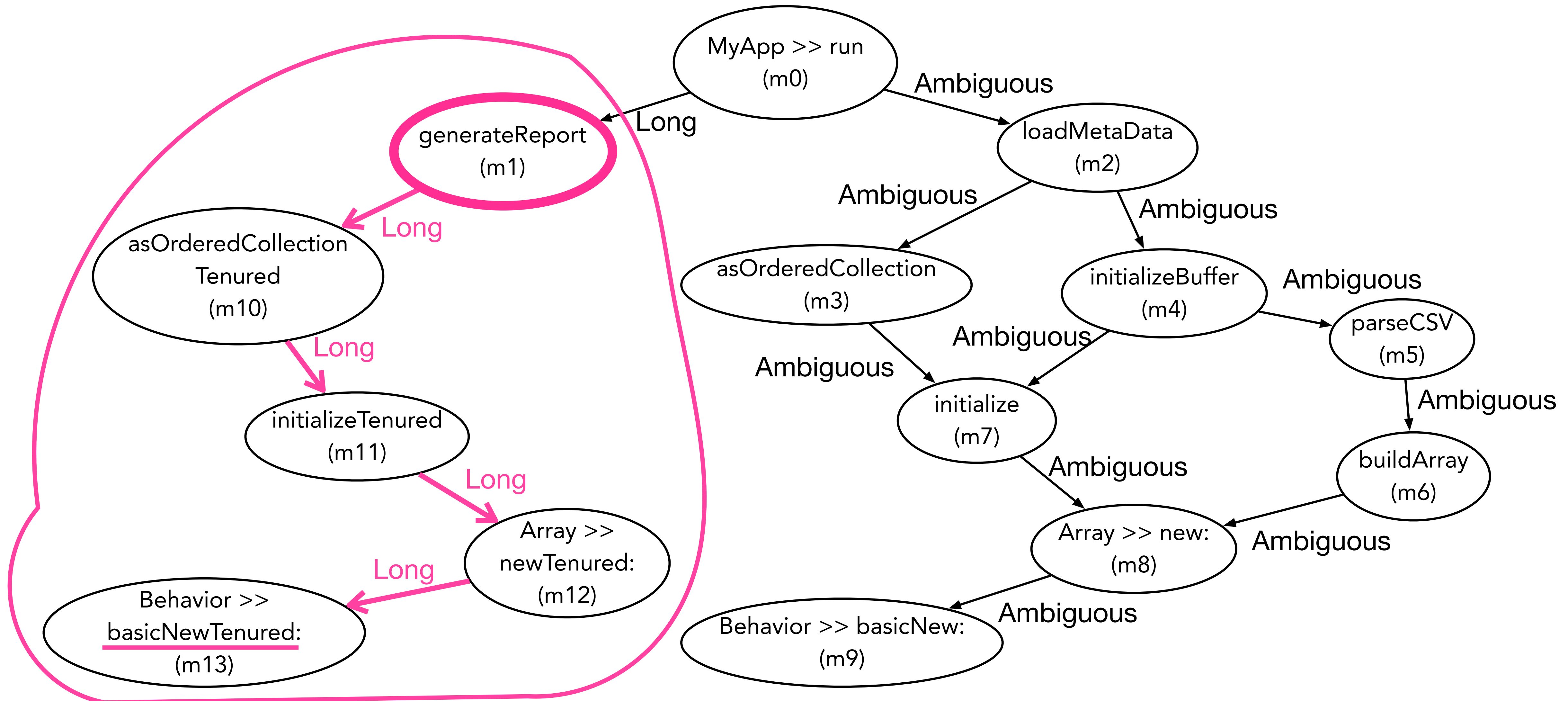
4. Graph Traversal



Allocation path

m1 → m3 → m7 → m8 → m9

5. Code Rewriting



PSP Research Questions

- **RQ.1 Performance.** Does PSP improve the performance of the textual allocation site?
- **RQ.2 GC Overhead.** How does the GC overhead of PSP compares to the textual allocation site?
- **RQ.3 JIT Compilation Overhead.** Does PSP has any impact on JIT compilation?

PSP Research Questions

*Focus of this
Presentation*

- **RQ.1 Performance.** Does PSP improve the performance of the textual allocation site?
- **RQ.2 GC Overhead.** How does the GC overhead of PSP compares to the textual allocation site?
- **RQ.3 JIT Compilation Overhead.** Does PSP has any impact on JIT compilation?

RQ.1 Performance

<1 is improvement; >1 is decrease

Cormas			
Strategy	0.1%	50%	100%
Textual	0.96	1.16	1.16
PSP70	1.19	0.97	0.98
PSP80	1.19	0.97	0.98
PSP90	1.18	0.97	0.98

DataFrame			
Strategy	0.1%	50%	100%
Textual	0.99	1.05	0.99
PSP70	0.89	0.92	0.93
PSP80	0.98	0.94	0.94
PSP90	1.22	0.93	0.95

- At high sampling rates, **PSP consistently outperforms** the textual strategy (peak improvement 0.89×).

RQ.2 Garbage Collection Overhead

<1 is improvement; >1 is decrease

Strategy	Cormas				DataFrame			
	0.1%	10%	50%	100%	0.1%	10%	50%	100%
Textual	0.68	0.52	0.54	0.49	0.06	5.58	5.59	5.59
PSP70	0.23	0.22	0.17	0.24	5.58	1.00	0.53	0.53
PSP80	0.29	0.24	0.24	0.24	5.58	0.99	0.53	0.53
PSP90	0.42	0.27	0.24	0.24	5.58	0.99	0.53	0.53

At high sampling rates:

- PSP reduces GC time up to **0.17x**
- PSP **consistently beats the textual strategy**
- Textual strategy increases GC time up to **5.59x**

PSP Summary

Key findings:

- Speedups of up to 0.89×
- GC overhead reduced up to 0.17×
- Fully compatible with JIT compiler.
- Profile at high sampling rates.

Not all applications benefit:

- Need a benchmarking phase first.
- Application needs to have sufficient long-lived objects

Outline

1. Introduction
2. State of the Art
3. Finalization-Based Lifetime Profiling
4. Textual Allocation Site Pretenuring
5. Path-Sensitive Pretenuring
- 6. MethodProxies**
7. Conclusion

2. Profiling

How to build effective finalization lifetime profilers?

3. Allocation Sites

How to identify where object allocations are created?

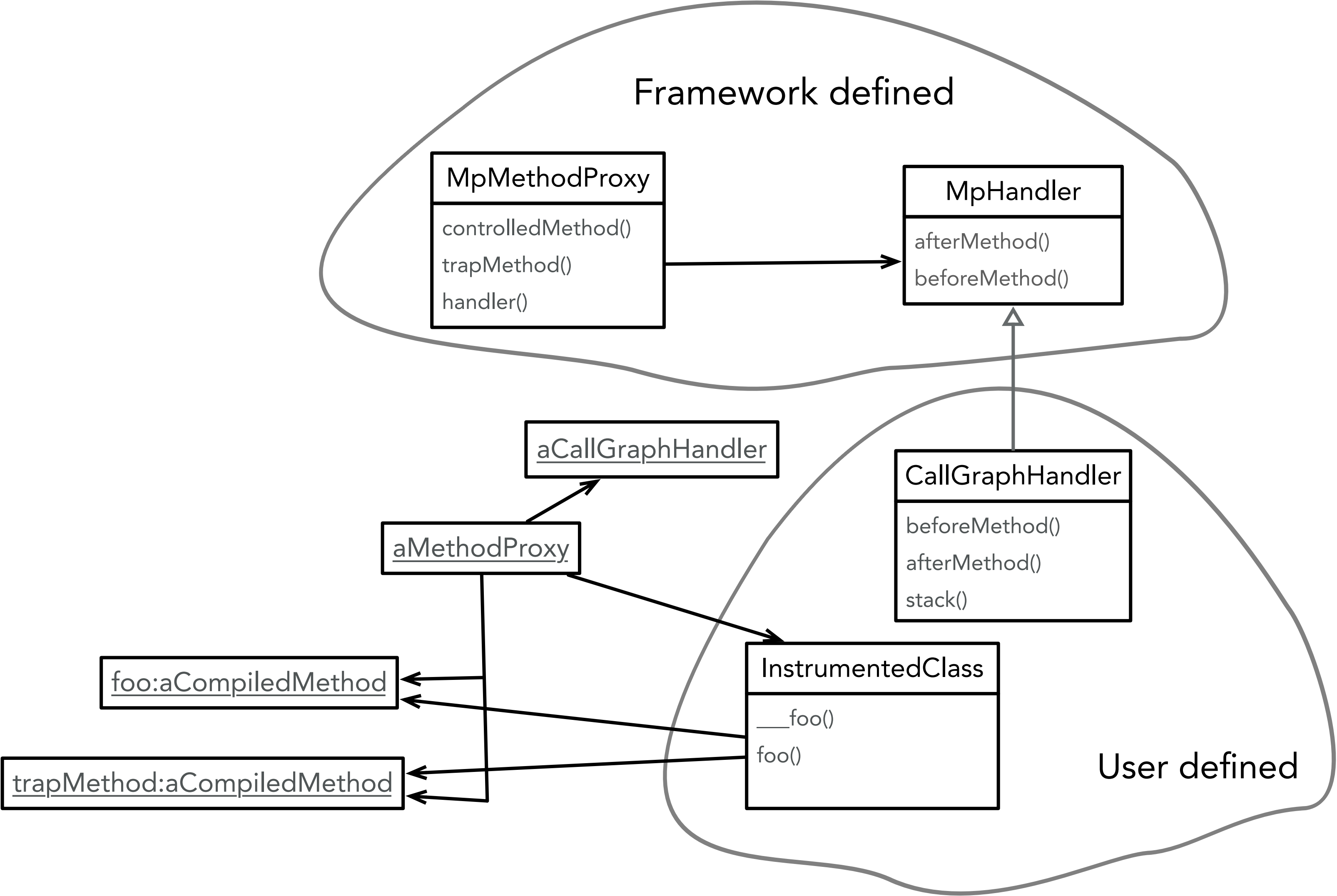
1. Instrumentation

How to instrument the code to capture object allocations?

Code Instrumentation Challenges

- **Meta-safety** — prevent infinite recursion when instrumented code calls instrumented methods
- **Unwind-safety** — handle exceptions and non-local returns
- **Meta-thread safety** — concurrent access to meta-state
- **Dynamic (de)instrumentation** — to disable instrumentation on-the-fly
- **Practical overhead** — the instrumentation should be usable

MethodProxies Stratified Design

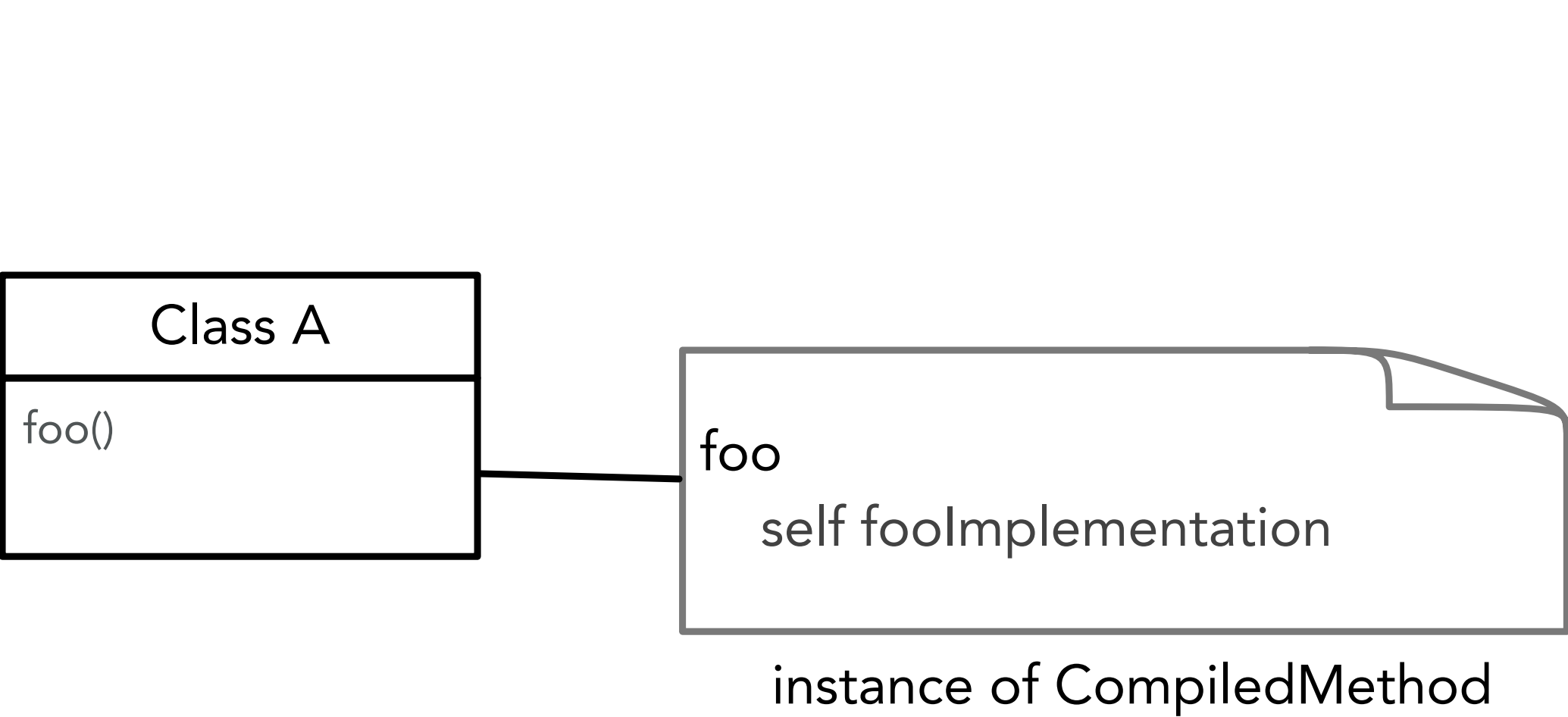


MethodProxies Principles

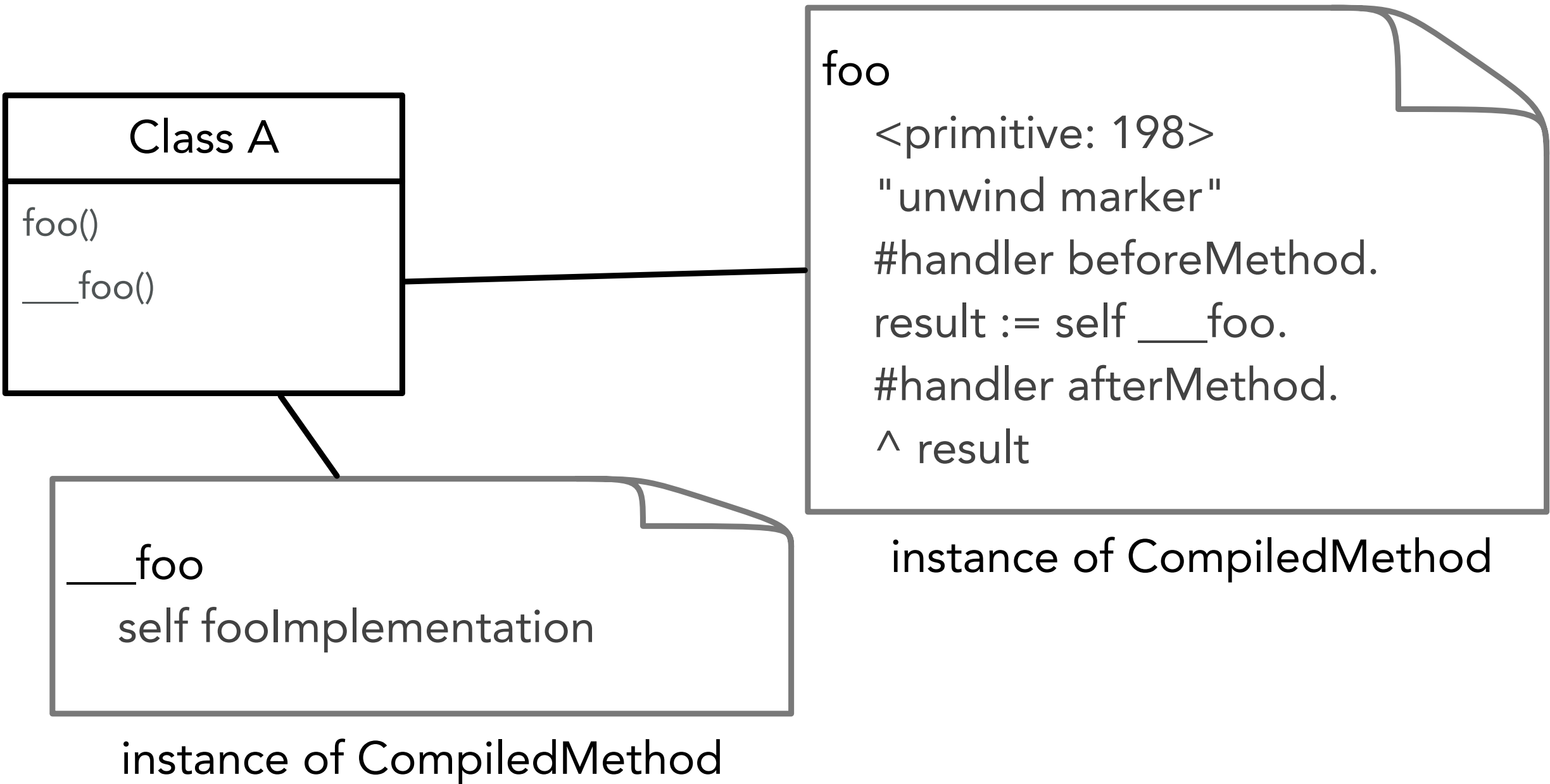
- Precompiled per-**method trap stubs**
- **Literal patching** for fast installation
- **Thread-local meta-safety**
- Optimized **unwind handling**
- **Dynamic (de)instrumentation**

MethodProxies Instrumentation

Before instrumentation



After instrumentation



Evaluation

Application	Mp Mean	Mp Range	Safety	Mp vs Mgc
DataFrame (617 methods)	1.12×	1.04–1.28×	1.07×	5.57×
Microdown (2,062 methods)	3.10×	1.36–8.51×	0.98×	8.81×
Cormas (2,135 methods)	1.48×	0.93–2.19×	1.31×	18.41×
Moose (11,009 methods)	1.10×	1.04–1.24×	1.02×	2.24×
Overall	1.54×		1.09×	6.47×

Average overhead: 1.54×.
Meta-safety cost: only 9%.

MethodProxies is on average 6.47×
faster than MessageGatekeeper,
with peaks up to 40.34×.

Outline

1. Introduction
2. State of the Art
3. Finalization-Based Lifetime Profiling
4. Textual Allocation Site Pretenuring
5. Path-Sensitive Pretenuring
- 6. Conclusion**

Thesis Key Findings

- Pure-OO languages break **textual allocation-site-based pretenuring**.
- Ephemeron-based finalization estimates **object lifetimes actionably** (1.26× overhead at 0.1% sampling).
- **Path-sensitive pretenuring** beats the textual strategy up to 0.89×.

Summary of Technical Contributions

1. Instrumentation



enables



Illimani

2. Profiling

enables



FiLiP

enables



PSP

3. Allocation Sites

 `pharo-contributions/MethodProxies`

 `jordanmontt/illimani-memory-profiler`

 `jordanmontt/path-sensitive-pretenuing`

 `jordanmontt/senders-chain-transformer`

- **MethodProxies** — Meta-safe, unwind-safe, JIT-compatible per-method instrumentation library.
- **Illimani** — Framework for building custom memory profilers.
- **FiLiP** — Actionable, practical overhead, consistent across sampling rates.
- **Path-Sensitive Pretenuing (PSP)** — Incorporates calling context to improve textual allocation strategy.

Summary of Scientific Contributions

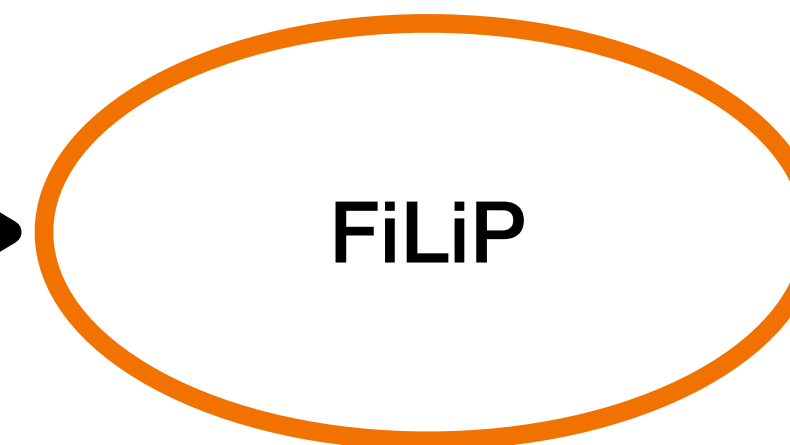
1. Instrumentation



2. Profiling



3. Allocation Sites



1. *Safe and Efficient Message-Passing Control* — **COLA 2026. Jornal paper**
2. *Insight: Towards a Human Usable Instrumentation Language* — IWSST 2026. Workshop paper
3. *MethodProxies: A Safe and Fast Message-Passing Control Library* — IWSST 2024. Workshop paper

4. *Evaluating Finalization-Based Object Lifetime Profiling* — **ISMM 2024. Conference paper**
5. *Improving Performance Through Object Lifetime Profiling: the DataFrame Case* — IWSST 2023. Workshop paper
6. *Garbage Collector Tuning in Pathological Allocation Pattern Applications* — IWSST 2023. Workshop paper

Future Work

Instrumentation:

- Statistical sampling at the instrumentation level
- Mitigate overprofiling and JIT perturbation
- Nested instrumentation

Profiling & Optimization:

- Strong actionability for finalization-based profiling
- Presizing, pretransitioning
- Adaptive parameter selection
- Broader evaluation

Summary and Takeaways

Pure-OO languages break classical pretenuring.

- All allocations collapse to a few methods, making textual allocation sites ineffective.

Ephemeron-based finalization is actionable.

- FiLiP estimates object lifetimes with a 1.26× overhead at 0.1% sampling rate with no VM modifications needed.

Path-sensitive pretenuring wins.

- Incorporating calling context into allocation site consistently outperforms the textual strategy.

Artifacts: MethodProxies · Illimani + FiLiP · Path-Sensitive Pretenuring

Contact:  github.com/jordanmontt · sebastianjmt@protonmail.ch

Merci :)